



第3章 ARM指令集

本章主要内容

- 1、ARM指令集简介；
- 2、ARM指令的寻址方式；
- 3、ARM指令集。

3.1 ARM指令集简介

ARM微处理器的**ARM指令集**，所有的指令长度都是**32位**，并且大多数指令都在一个单独指令周期内执行。

主要特点包括：

- 指令是条件执行的;
- ARM微处理器的指令集是加载/存储型的;
- 在多寄存器操作指令中一次最多可以完成16个寄存器的数据传送。

3.1 ARM指令集简介

- 3.1.1 ARM指令格式
- 3.1.2 ARM指令的条件码
- 3.1.3 ARM指令分类

3.1.1 ARM指令格式

一、用助记符表示的ARM指令一般格式如下：

<opcode> {<cond>} {S} {<Rd>} { , <Rn>} { , <OP2>}

格式中< >的内容必不可少，{ }中的内容可省略。

<opcode>表示操作码，如ADD表示算术加法。

{<cond>}表示指令执行的条件域，如EQ、NE等。

{S}决定指令的执行结果是否影响CPSR的值，使用该后缀则指令执行的结将果影响CPSR的值，否则不影响。

3.1.1 ARM指令格式

<opcode> {<cond>} {S} {<Rd>} { , <Rn>} { , <OP2>}

<Rd>表示目标或源寄存器。

<Rn>表示第一个操作数，为寄存器。

<op2>表示第二个操作数，可以是立即数、寄存器和寄存器移位操作数。

除了<opcode>其余域都可以选择使用。

3.1.1 ARM指令格式

例如：指令ADDEQS R0,R1,#8;

操作码为ADD,

条件域cond为EQ,

S表示该指令的执行影响CPSR寄存器的值,

目的寄存器Rd为R0,

第一个操作数寄存器Rn为R1,

第二个操作数OP2为立即数 # 8。

执行结果：R0 = R1 + 8

3.2.1指令和指令格式

指令格式举例：

LDR R3, [R6]

将R6寄存器内容所指向的存储单元内容读出，保存在R3中，
执行条件AL。

BEQ NEXT

分支指令B，加上后缀EQ，变为BEQ，表示“满足条件相等则跳转”。



ADD R0, R1, #2

加法指令ADD, $R0=R1+2$, 不带S, 计算结果不影响CPSR寄存器。

SUBNES R0, R1, #0X03

条件执行减法运算 (NE), $R0=R1-0X03$, 影响CPSR寄存器。

3.1.1 ARM指令格式

二、指令的机器码

从形式上看，ARM指令在机器中的表示格式是用32位的二进制数表示。如ARM中有一条指令为：

ADDEQS R0, R1, #8

其二进制代码形式为：

31~28	27~25	24~21	20	19~16	15~12	11~0
0000	001	0100	1	0001	0000	000000001000
cond		opcode	s	Rn	Rd	Op2

3.1.1 ARM指令格式

$\langle \text{opcode} \rangle \{ \langle \text{cond} \rangle \} \{ S \} \langle \text{Rd} \rangle, \langle \text{Rn} \rangle \{, \langle \text{OP2} \rangle \}$

31~28	27~25	24~21	20	19~16	15~12	11~0
0000	001	0100	1	0001	0000	000000001000
cond		opcode	s	Rn	Rd	Op2

ARM指令代码一般可以分为5个域:

- 第1个域是4位[31:28]的条件码域, 4位条件码共有16种组合;
- 第2个域是指令代码域[27:20],除了指令编码外, 还包含几个很重要的指令特征和可选后缀的编码;

3.1.1 ARM指令格式

<opcode> {<cond>} {S}<Rd>, <Rn> {, <OP2>}

31~28	27~25	24~21	20	19~16	15~12	11~0
0000	001	0100	1	0001	0000	000000001000
cond		opcode	s	Rn	Rd	Op2

- 第3个域是第1个操作数寄存器Rn，是4位[19:16],为R0 ~ R15共16个寄存器编码；
- 第4个域是目标或源寄存器Rd，是4位[15:12],为R0 ~ R15共16个寄存器编码；
- 第5个域是第二个操作数[11:0]。

3.1.1 ARM指令格式

三、指令的可选后缀

1) S后缀

指令中使用S后缀时，指令执行后程序状态寄存器的条件标志位将被刷新，不使用S后缀时，指令执行后程序状态寄存器CPSR的条件标志将不会发生变化。

3.1.1 ARM指令格式

例:假设R0=0x1,R3=0x3,指令执行之前CPSR部分标志位为nZcv,分别执行如下指令CPSR的值有何变化?

SUB R1,R0,R3 ; R0的值减去R3的值, 结果存入R1

SUBS R1,R0,R3 ; R0的值减去R3的值, 结果存入R1
; 影响标志位。

分析: 执行第1条指令对于标志寄存器的值没有任何影响, 因此CPSR的值不变。执行第2条指令后CPSR=Nzcv。

3.1.1 ARM指令格式

2) !后缀

如果指令地址表达式中不含! 后缀, 则基址寄存器中的地址值不会发生变化。

指令中的地址表达式中含有!后缀时, 指令执行后, 基址寄存器中的地址值将发生变化, 变化的结果如下:

基址寄存器中的值 (指令执行后) = 指令执行前的值 + 地址偏移量

3.1.1 ARM指令格式

例 分别执行下面两条指令有何区别？

LDR R3,[R0,#4]

LDR R3,[R0,#4]!

分析：在上述指令中，第1条指令没有后缀！，指令的结果是把R0加4作为地址指针，把这个指针所指向的地址单元所存储的数据读入R3，R0的值不变。第2条指令除了实现以上操作外，还把 $R0 + 4$ 的结果送到R0中。

3.1.1 ARM指令格式

使用!后缀需要注意如下事项:

- (1)! 后缀必须紧跟在地址表达式后面, 而地址表达式要有明确的地址偏移量;
- (2)! 后缀不能用于R15(PC)的后面;
- (3)当用在单个地址寄存器后面时, 必须确信这个寄存器有隐性的偏移量, 例如 “STMIA R7!, {R0 – R3}” 此时地址基址寄存器R7的隐性偏移量是16字节。如果R7的初始值为 0X40000000, 则该语句结束后为0X40000010

3.1.1 ARM指令格式

3) B后缀

B后缀的含义是：指令所涉及的数据是一个字节，不是一个字或半字。

LDR R4, [R0]

R4=[R0], 指令传送一个字

LDR**B** R4, [R0]

R4=[R0], 指令传送一个字节

LDRE**QB** R4, [R0]

如果相等则执行, R4=[R0], 指令传送一个字节

3.1.2 ARM指令的条件码

当处理器工作在 ARM状态 时，几乎所有的指令均能根据CPSR中的条件标志位的状态和指令的条件域 有条件地执行。当指令的执行条件满足时，指令被执行，否则指令被忽略。

条件后缀只是影响指令是否执行，不影响指令的内容。

3.1.2 ARM指令的条件码

ARM 指令的 **条件码** 和 **助记符** 如下表所示:

条件码	条件码助记符	CPSR中条件标志位值	含义
0000	EQ	Z=1	相等
0001	NE	Z=0	不相等
0010	CS/HS	C=1	无符号数大于或等于
0011	CC/LO	C=0	无符号数小于
0100	MI	N=1	负数
0101	PL	N=0	正数或零
0110	VS	V=1	溢出
0111	VC	V=0	未溢出

3.1.2 ARM指令的条件码

条件码	条件码助记符	CPSR中条件标志位值	含义
1000	HI	$C=1$ 且 $Z=0$	无符号数大于
1001	LS	$C=0$ 或 $Z=1$	无符号数小于或等于
1010	GE	$N=V$	带符号数大于或等于
1011	LT	$N \neq V$	带符号数小于
1100	GT	$Z=0$ 且 $N=V$	带符号数大于
1101	LE	$Z=1$ 或 $N \neq V$	带符号数小于或等于
1110	AL		无条件执行
1111	NV	ARMV3之前	从不执行(不要使用)

3.1.2 ARM指令的条件码

条件后缀和S后缀的关系如下：

(1) 如果既有条件后缀又有S后缀，则书写时S排在后面

如：ADDEQS R1,R0,R2

该指令在Z=1时执行，将R0+R2的值放入R1，同时刷新条件标志位。

(2) 条件后缀是要测试条件标志位，而S后缀是要刷新条件标志位。

(3) 条件后缀要测试的是执行前的标志位，而S后缀是依据指令的结果改变条件标志。

3.1.2 ARM指令的条件码

例：下面三条指令有何区别？

ADD R4,R3,#1

ADDEQ R4,R3,#1

ADDS R4,R3,#1

3.1.3 ARM指令分类

ARM 指令可以分为：分支指令、数据处理指令、存储访问指令、协处理器指令和杂项指令五类。

① 分支指令

分支指令用于控制程序的执行流程、实现ARM代码与Thumb代码之间进行切换。

② 数据处理指令

数据处理指令在通用寄存器上执行计算，主要分为3种：算术/逻辑指令、比较指令和乘法指令。

3.1.3 ARM指令分类

③ 存储访问指令

用于加载/存储存放于MCU片外存储系统中的数据。加载指令用于从内存中读取数据放入寄存器中，存储指令用于将寄存器中的数据保存到内存中。

3.1.3 ARM指令分类

④ ARM协处理器指令

ARM协处理器指令用于控制外部的协处理器。包括

- 数据处理指令：启动一个协处理器专用的内部操作。
- 数据转移指令：使数据在协处理器和存储器之间进行转移。
- 寄存器转移指令：协处理器值转移到ARM寄存器或ARM寄存器的值转移到协处理器。

3.1.3 ARM指令分类

⑤ 杂项指令

包括状态寄存器转移指令和异常中断产生指令。

- 状态寄存器转移指令将CPSR或SPSR的内容转移到一个通用寄存器，或者反过来将通用寄存器的内容写入CPSR或SPSR寄存器
- ARM有 两条 异常中断产生指令，分别为 软中断指令SWI 和 断点中断指令BKPT。

3.2 指令的寻址方式

- 3.2.1 立即数寻址
- 3.2.2 寄存器寻址
- 3.2.3 寄存器移位寻址
- 3.2.4 寄存器间接寻址
- 3.2.5 基址变址寻址
- 3.2.6 多寄存器寻址
- 3.2.7 堆栈寻址
- 3.2.8 相对寻址

3.2.1立即数寻址

立即数寻址也叫立即寻址，操作数本身就在指令中给出，取出指令也就取到了操作数。这个操作数被称为立即数，对应的寻址方式也就叫做立即数寻址。

- 立即数要求以 “#” 为前缀，
 - 对于以十六进制表示的立即数，还要求在 “#” 后加上 “0x” 或 “&” ；
 - 对于以二进制表示的立即数，要求在 “#” 后加上 “0b” ；
 - 对于以十进制表示的立即数，要求在 “#” 后加上 “0d” 或缺省。

3.2.1立即数寻址

- 例1 ADD R0, R0, #1 ; $R0 \leftarrow R0 + 1$
- 例2 MOV R0, #15 ; $R0 \leftarrow \#15$

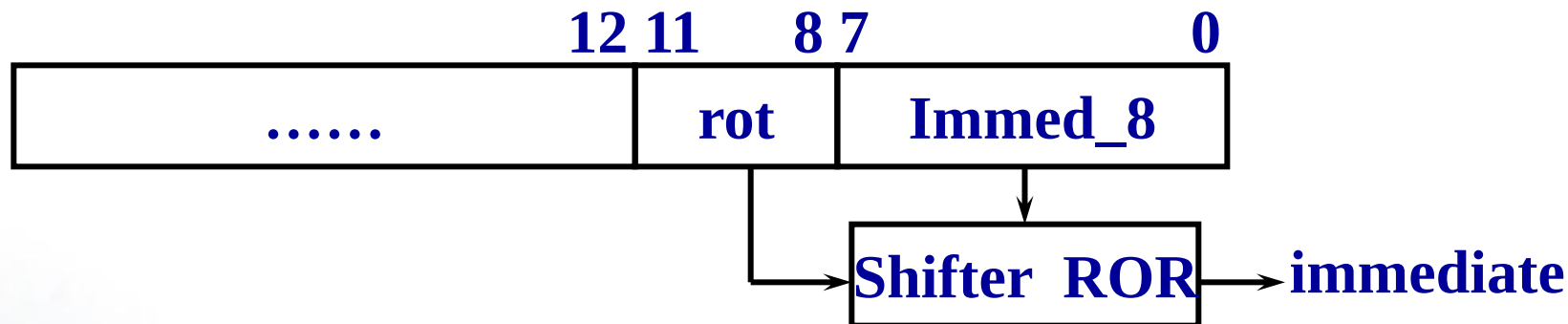
此指令是将立即数15传送到R0中，该指令的机器编码为E3A0000FH，由指令编码中0x00F即可得到一个32位的立即0x0000000FH，这个数再送入R0



如何构造32位立即数？

3.2.1立即数寻址

在指令格式中，第二个操作数有12位：



因此有效立即数immediate可以表示成：

$\langle \text{immediate} \rangle = \text{immed_8} \text{ 循环右移 } (2 \times \text{rot})$

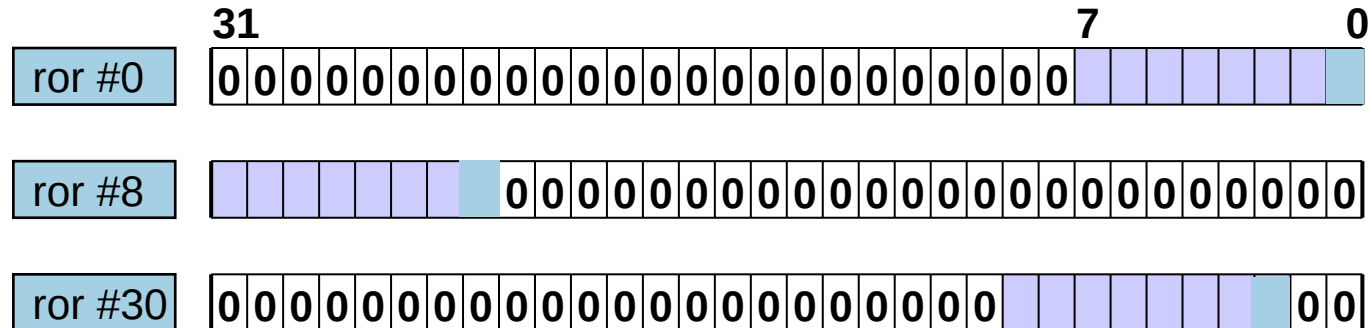
4 bit 移位因子 (0-15) 乘以2，得到一个范围在0-30，步长为2的移位值。

因此，将ARM中的立即数称为8位位图。

“最后8位移动偶数位” 得到立即数。

3.2.1立即数寻址

例:



下列命令中，汇编器把立即数转换为移位操作：

MOV R0,#0x104 ; uses 0x41 ror 30

ADD R1,R2,#0xFF0000 ; uses 0xFF ror 16

带有立即数的MOV 指令的二进制编码为：

MOV R0, #0xF200 ;E3A00CF2. 0xF200 =0xF2 ROR(2*C)

MOV R1, #0x110000 ;E3A01811. 0x110000 =0x11 ROR(2*8)

MOV R4, #0x12800 ;E3A04B4A. 0x12800 =0x4A ROR(2*B)

3.2.1立即数寻址

只有能够通过此构造方法得到的才是合法的立即数。

- 合法立即数：0xFF; 0x104(8位图为0x41); 0xFF0; 0xFF00
- 非法立即数：0x101; 0x102; 0xFF1

深入理解：一个合法的立即数可能有多种编码方法，将使某些指令的执行产生不同的结果。 如

0x3F0  $\begin{cases} \text{immed_8}=0x3F, \text{ rot}=0xE, \text{ 对}3F\text{左移}4\text{位} \\ \text{immed_8}=0xFC, \text{ rot}=0xF, \text{ 对}FC\text{左移}2\text{位} \end{cases}$

ARM汇编编译器生成立即数的规则为：

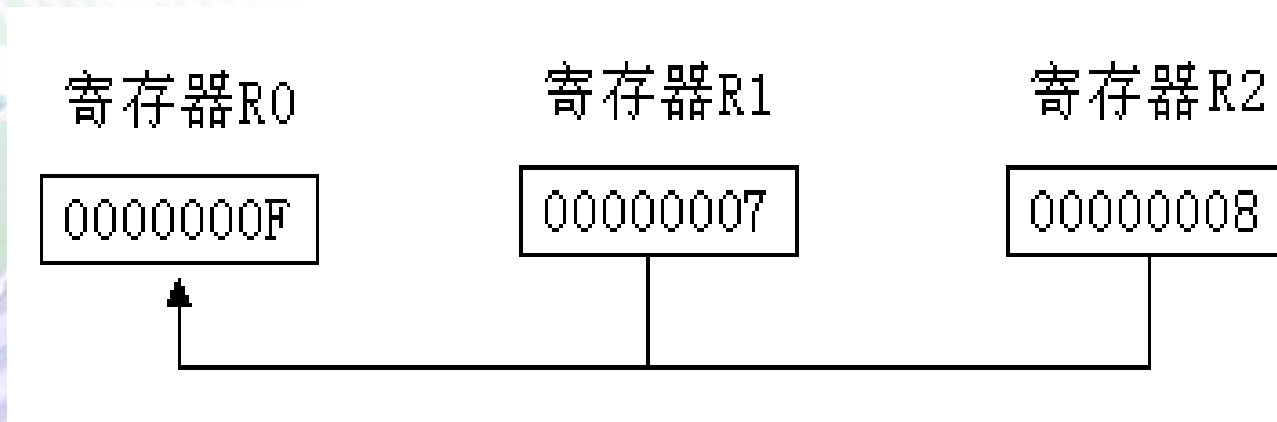
- 当立即数数值在0到0xFF范围时，令
 $\text{immed_8} = \langle \text{immediate} \rangle, \text{ rot} = 0。$
- 其它情况下，汇编编译器选择使rot数值最小的编码方式。

3.2.2寄存器寻址

寄存器寻址就是利用寄存器中的数值作为操作数，这种寻址方式是各类微处理器经常采用的一种方式，也是一种执行效率较高的寻址方式。

例： MOV R1, R2 ; R1 $\leftarrow$ R2

ADD R0, R1, R2 ; R0 $\leftarrow$ R1 + R2



3.2.3 寄存器移位寻址

一、寄存器移位寻址

当第二操作数为寄存器型时，在执行寄存器寻址操作时，也可以对第二操作数寄存器进行移位，此时第二操作数形式为：

ADD Rd, Rn, Rm, {<shift>}

其中：

- Rm 称为第二操作数寄存器
- <shift> 用来指定移位类型和移位位数，有两种形式：
 - 5位立即数 (其值小于32)
 - 寄存器(用Rs表示) (其值小于32)

3.2.3 寄存器移位寻址

在指令执行时，将寄存器**移位后**的内容，作为第二操作数参与运算。例如指令：

ADD R3, R2, R1, LSR #2 ; $R3 \leftarrow R2 + (R1 \text{右移} 2 \text{位})$

ADD R3, R2, R1, LSR R0 ; $R3 \leftarrow R2 + (R1 \text{右移} R0 \text{位})$

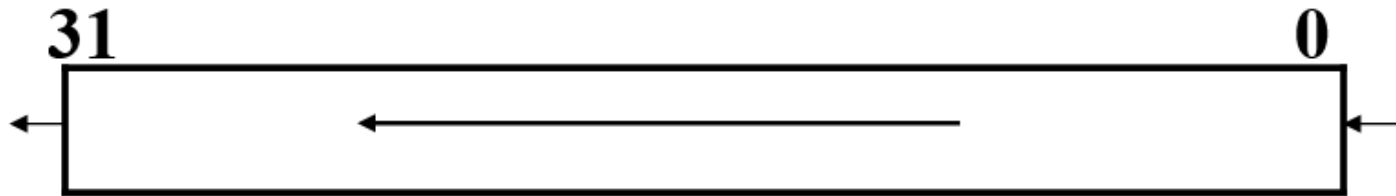
二、第二操作数移位方式

共有6种移位方式：

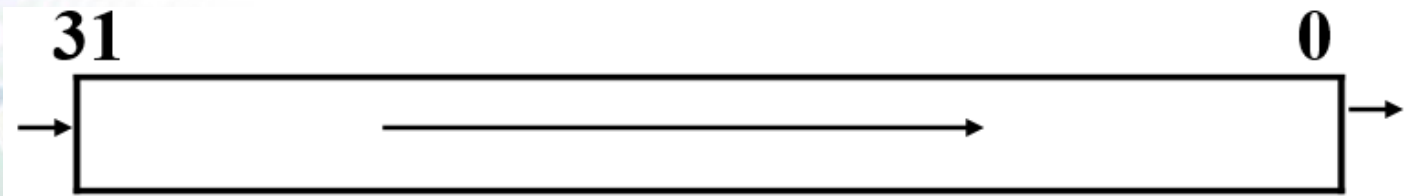
- LSL 逻辑左移
- ASL 算术左移
- ROR 循环右移
- LSR 逻辑右移
- ASR 算术右移
- RRX 带扩展的循环右移

3.2.3 寄存器移位寻址

(1) LSL: 逻辑左移, 空出的最低有效位用0填充。



(2) LSR: 逻辑右移, 空出的最高有效位用0填充。



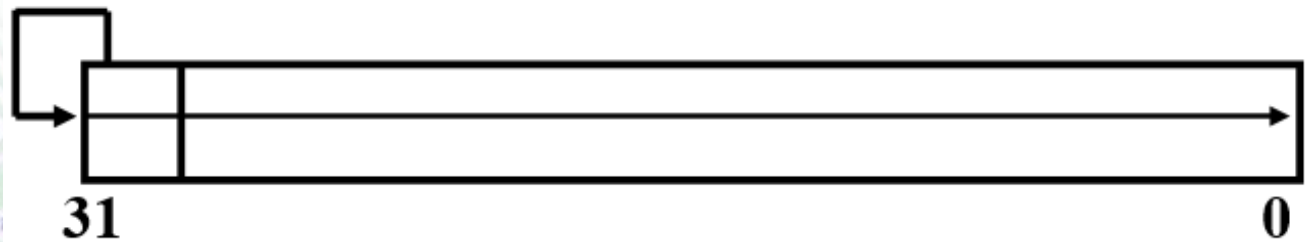
SUB R3, R2, R1, LSL #2 ; R3←R2-(R1左移2位)

SUB R3, R2, R1, LSR R0 ; R3←R2-(R1右移R0位)

3.2.3 寄存器移位寻址

(3) ASL: 算术左移, 由于左移空出的有效位用0填充, 因此它与LSL同义。

(4) ASR: 算术右移 (Arithmetic Shift Right)。算术移位的对象是带符号数, 移位过程中必须保持操作数的符号不变。如果源操作数是正数, 空出的最高有效位用0填充, 如果是负数用1填充。

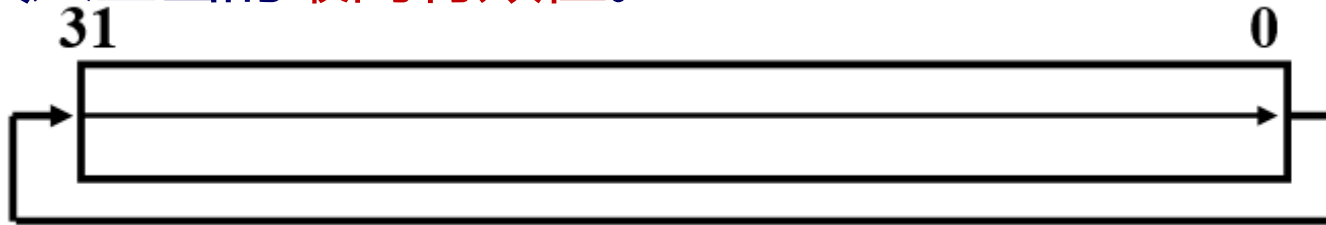


ADD R3, R2, R1, ASL #2 ; $R3 \leftarrow R2 + (R1 \text{ 左移 } 2 \text{ 位})$

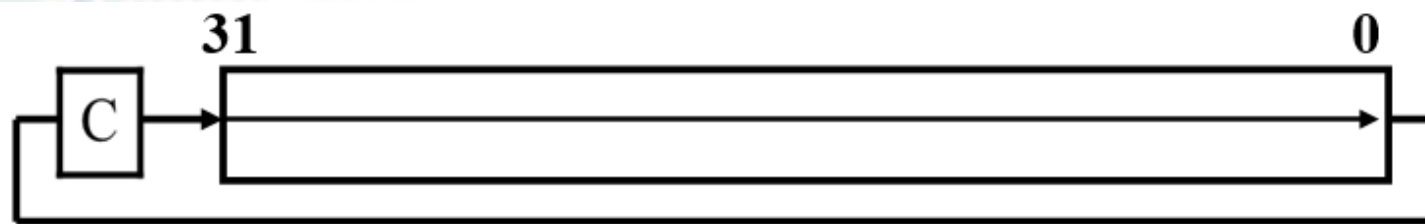
SUB R3, R2, R1, ASR R3 ; $R3 \leftarrow R2 - (R1 \text{ 算术右移 } R3 \text{ 位})$

3.2.3 寄存器移位寻址

(5) ROR: 循环右移(Rotate Right), 移出的字的最低有效位依次填入空出的最高有效位。



(6) RRX: 带进位位的循环右移(Rotate Right Extended)。将寄存器的内容循环右移1位, 空位用原来C标志位填充。



SUB R3, R2, R1, ROR #2 ; R3←R2-(R1循环右移2位)

SUB R3, R2, R1, RRX

; R3←R2-(R1带进位位循环右移1位)

3.2.3寄存器移位寻址

三、第二操作数的移位位数

移位位数可以用立即数方式或者寄存器方式给出，其值均小于32，应为0---31。

如下所示：

ADD R3, R2, R1, LSR #2 ; R3=R2+(R1右移2位)

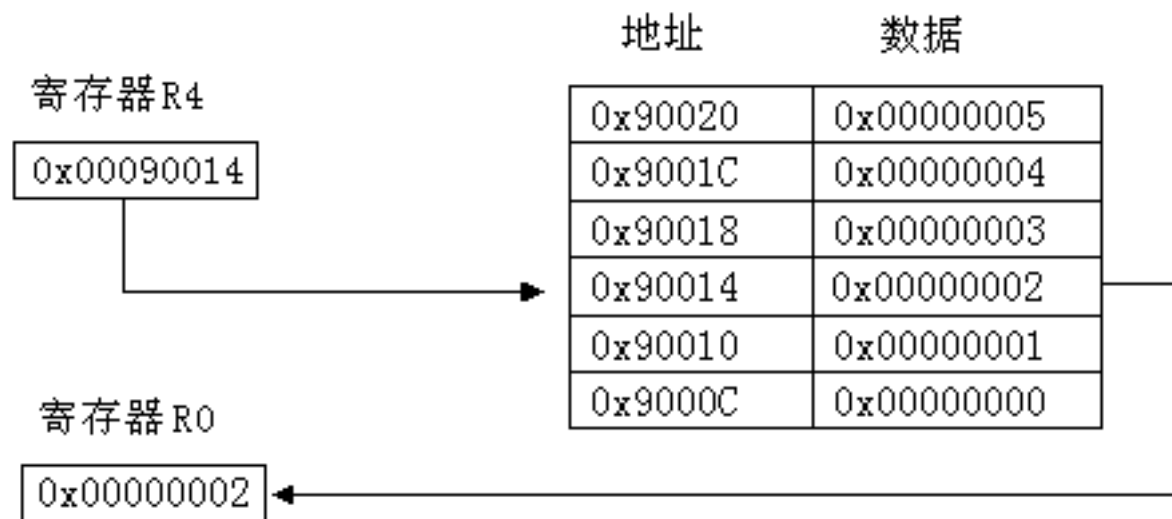
ADD R3, R2, R1, LSR R4 ; R3=R2+(R1右移R4位)

寄存器R1的内容分别逻辑右移2位、R4位,再与寄存器R2的内容相加，结果放入R3中。

3.2.4 寄存器间接寻址

寄存器间接寻址就是以寄存器中的值作为操作数的地址，而操作数本身存放在存储器中。

例：LDR R0, [R4] ; R0 ← [R4]



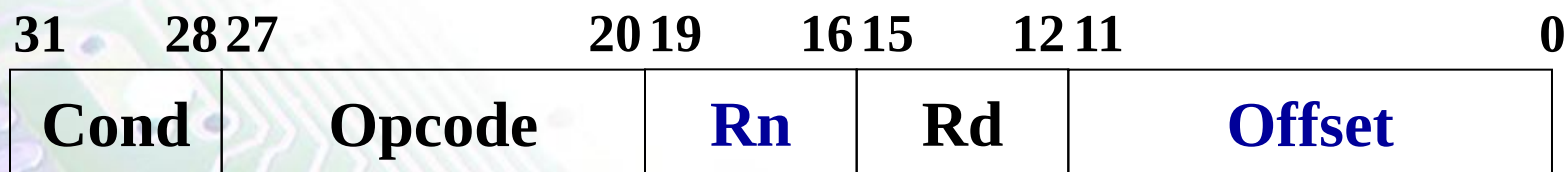
STR R0, [R4] ; R0 → [R4]

3.2.5 基址变址寻址

一、变址寻址

也叫**基址变址寻址**：将**基址寄存器**的内容与指令中给出的**地址偏移量**相加，得到操作数所在的存储器的**有效地址**。

变址寻址方式常用于访问某基地址**附近**的地址单元。（4K范围的偏移）



例如：

LDR R0, [R1, # 4] ; $R0 \leftarrow \text{mem32}[R1 + 4]$

3.2.5 基址变址寻址

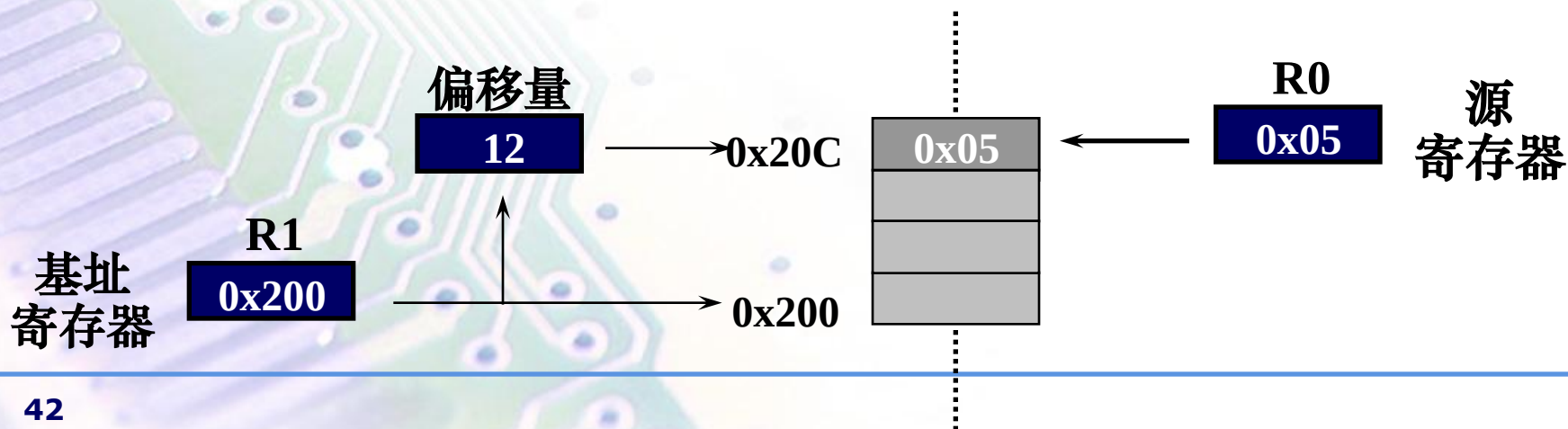
二、偏移地址方式

有三种加偏址的方式：前变址、自动变址和后变址寻址方式。

1、前变址模式（不修改基址寄存器）：

先基址+偏址，生成操作数地址，再做指令指定的操作。
也叫前索引偏移。

Pre-indexed: STR r0,[r1,#12]



3.2.5 基址变址寻址

2、自动变址模式(修改基址寄存器):

- ①先基址+偏移，生成操作数地址，做指令指定的操作。
- ②然后自动修改基址寄存器。

例如：

```
LDR R0, [R1, # 4]!    ; R0←mem32 [R1 + 4]  
                        ; R1←R1 + 4
```

！ ——表示更新基址寄存器。

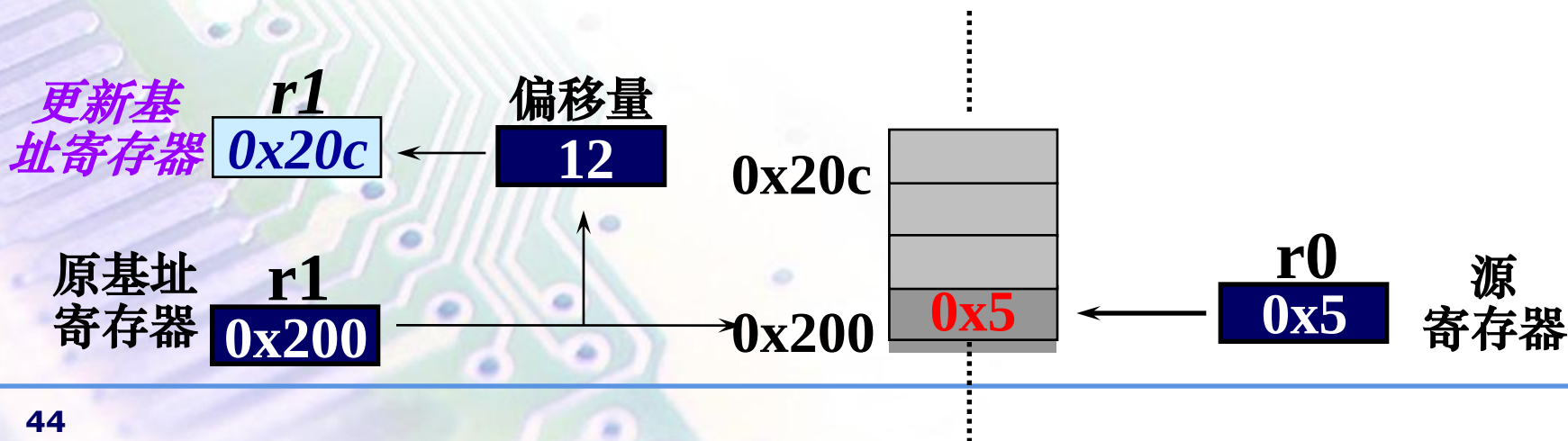
3.2.5 基址变址寻址

3、后变址模式(修改基址寄存器):

- ①基址寄存器不加偏移作为操作数地址。
- ②完成指令操作后，用(基址+偏移)的值修改基址寄存器。

即先用基地址传数，然后修改基地址（基址+偏移），也叫后索引偏移。

例 Post-indexed: STR r0,[r1],#12



3.2.5 基址变址寻址

三、偏移地址形式

可以是一个立即数，也可以是另一个寄存器，并且还可以是寄存器移位操作。如下所示：

LDR R2, [R3, #0X0C] ;R2 ← [R3+0X0C]

STR R1, [R0, #-0x4]! ;R1 → mem32[R0-4],
 ;R0=R0-4

LDR r0, [r1, r2] ;r0 ← mem32[r1+r2]

LDR r0, [r1, r2, LSL #2] ;r0 ← mem32[r1+r2*4]

常用的是立即数偏移的形式。

3.2.5 基址变址寻址

使用**后变址**寻址方式完成**表复制**程序：

COPY ADR R1, NEXT1 ;R1指向NEXT1

ADR R2, NEXT2 ;R2指向NEXT2

LOOP LDR R0, [R1],#4 ;取一个数

STR R0, [R2],#4 ;复制一个数

.....

NEXT1 ;源数据

.....

NEXT2 ;目标数据

3.2.6多寄存器寻址

采用多寄存器寻址方式，一条指令可以完成多个寄存器值的传送。

这种寻址方式是多寄存器传送指令LDM/STM的寻址方式，这种寻址方式中用一条指令最多可传送16个通用寄存器的值。连续的寄存器间用“-”连接，否则用“，”分隔。

例 LDMIA R0! , {R1-R4}

; $R1 \leftarrow [R0]$ 、 $R2 \leftarrow [R0 + 4]$ 、 $R3 \leftarrow [R0 + 8]$ 、 $R4 \leftarrow [R0 + 12]$

注意：对于所有LDM/STM指令而言，寄存器序号低的，在低地址单元，序号大的在高地址单元！与书写顺序无关！

3.2.6多寄存器寻址

	地址	数据	
	0x90020	0x00000005	
	0x9001C	0x00000004	
	0x90018	0x00000003	
	0x90014	0x00000002	
R0=0x90010 →	0x90010	0x00000001	
	0x9000C	0x00000000	

LDMIA R0! , {R1-R4}

指令执行前内存和寄存器值

	地址	数据	
R0=0x90020 →	0x90020	0x00000005	
	0x9001C	0x00000004	→ R4=0x00000004
	0x90018	0x00000003	→ R3=0x00000003
	0x90014	0x00000002	→ R2=0x00000002
	0x90010	0x00000001	→ R1=0x00000001
	0x9000C	0x00000000	

指令执行后内存和寄存器值

3.2.6多寄存器寻址

4种寻址操作

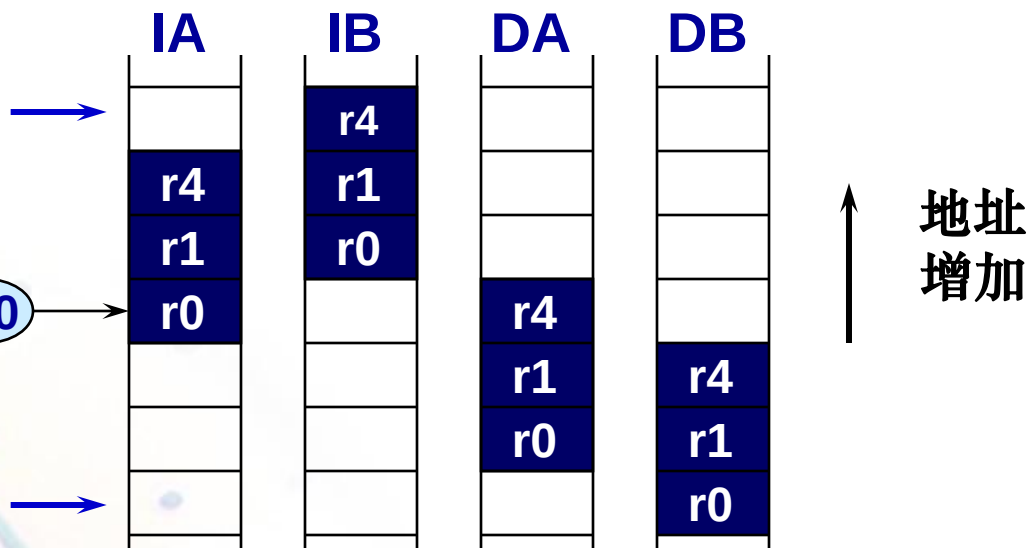
LDMIA/STMIA	Increment After(先传送, 后地址加4)
LDMIB/STMIB	Increment Before(先地址加4, 后传送)
LDMDA/STMDA	Decrement After(先传送, 后地址减4)
LDMDB/STMDB	Decrement Before(先地址减4, 后传送)

STMxx r10! , {r1,r4,r0}

地址递增, 指针
最后位置

基址寄存器 (Rn) r10

地址递减, 指针
最后位置



3.2.7 堆栈寻址

堆栈是一种数据结构，按**后进先出**（Last In First Out, LIFO）的方式工作，使用一个称作**堆栈指针**的专用寄存器指示当前的操作位置，堆栈指针总是指向**栈顶**。

3.2.7 堆栈寻址

➤ 堆栈可分为两种**增长方式**:

- **向上生长**: 向**高地址**方向生长, 称为**递增堆栈**。
- **向下生长**: 向**低地址**方向生长, 称为**递减堆栈**。

➤ 根据堆栈指针指向的**数据位置**的不同, 可分为:

- **满堆栈**: 堆栈指针指向**最后压入**堆栈的**有效数据项**, 称为满堆栈;
- **空堆栈**: 堆栈指针指向**下一个待压入数据**的空位置, 称为空堆栈。

这样就有4种类型的堆栈表示递增和递减的满和空堆栈的各种组合。

3.2.7 堆栈寻址

四种类型的堆栈工作方式

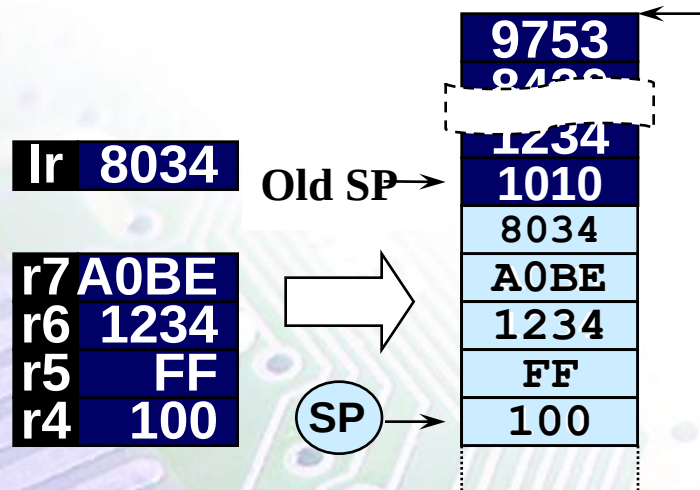
- **满递增堆栈**FA(Full Ascending): 堆栈指针指向最后压入的数据, 且由低地址向高地址生长。
- **满递减堆栈**FD(Full Descending) : 堆栈指针指向最后压入的数据, 且由高地址向低地址生长。
- **空递增堆栈**EA(Empty Ascending): 堆栈指针指向下一个将要放入数据的空位置, 且由低地址向高地址生长。
- **空递减堆栈**ED(Empty Descending): 堆栈指针指向下一个将要放入数据的空位置, 且由高地址向低地址生长。

3.2.7 堆栈寻址

堆栈的四种类型的“递增”和“递减”都是相对入栈操作而言的！

STMFD sp!,{r4-r7,lr}

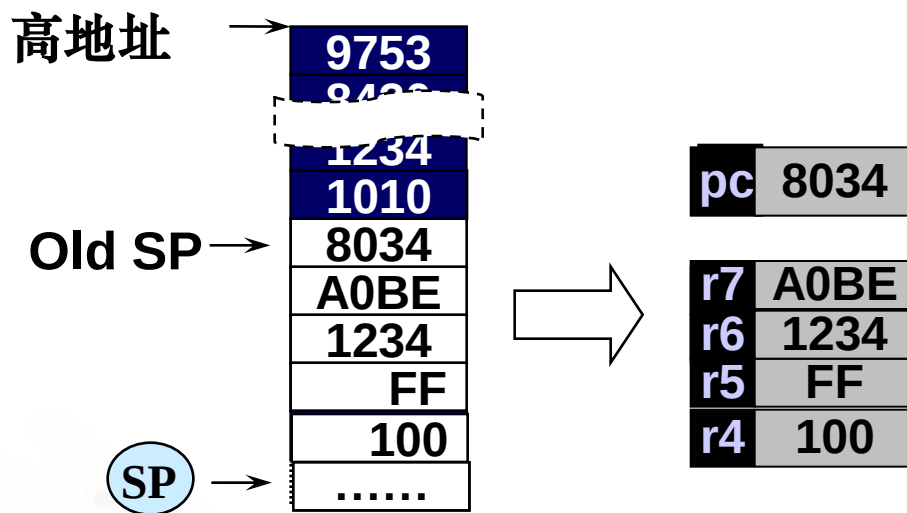
满递减



STMXX是存入到主存，是入栈操作

LDMFA sp!,{r4-r7,pc}

满递增



LDMXX是从主存读出到寄存器，是出栈操作

3.2.8 相对寻址

与基址变址寻址方式相类似，相对寻址以程序计数器PC的当前值为基地址，指令中的地址标号作为偏移量，将两者相加之后得到操作数的有效地址。

相对寻址指令举例如下：

```
BL    SUBR1          ; 调用到SUBR1子程序
      ...             ; SUBR1经编译后为24位有符号数
SUBR1  ... ..
      ... ..
MOV   PC, LR         ; 返回
```

3.3 ARM指令集

3.3.1 分支指令

3.3.2 数据处理指令

3.3.3 存储器访问指令

3.3.4 协处理器指令

3.3.5 杂项指令

3.3.1 分支指令

在ARM中有两种方式可以实现程序的跳转：

- 一种是使用分支转移指令直接跳转；
- 另一种则是直接向PC寄存器赋值来实现跳转。

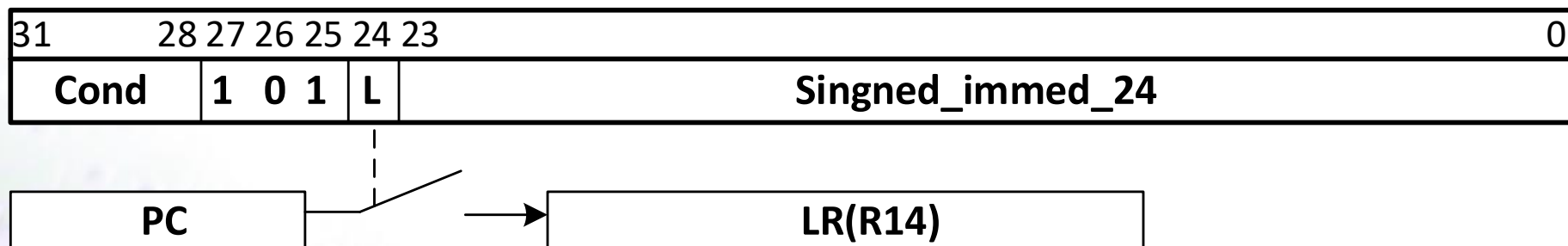
ARM的分支转移指令，可以从当前指令向前或向后的32MB的地址空间跳转，根据完成的功能它可以分为以下4种：

助记符	说明	操作
B	跳转指令	$PC \leftarrow label$
BL	带返回连接的跳转	$PC \leftarrow label$ (LR $\leftarrow$ BL后面的第一条指令地址)
BX	跳转并切换状态	$PC \leftarrow Rm \ \& \ 0xFFFFFFFFE, T \leftarrow Rm[0]$
BLX	带返回连接的跳转并切换状态	$PC \leftarrow label, T \leftarrow 1;$ $PC \leftarrow Rm \ \& \ 0xFFFFFFFFE, T \leftarrow Rm[0];$ LR $\leftarrow$ BL后面的第一条指令地址

3.3.1 分支指令

一、跳转指令 B 及带连接的跳转指令 BL

B 指令与 BL 指令的编码格式如下：



从编码中看到**L**控制了PC与LR寄存器之间的开关。当**L=0**时，该开关断开，指令为**B指令**；当**L=1**时，该开关接通，指令为**BL指令**。

Signed_immed_24表示24位**有符号**的立即数（偏移量）

3.3.1 分支指令

B 和 BL 指令的助记符格式为：

B{<cond>} <target> 和 BL{<cond>} <target>

- cond表示指令执行条件
- target表示跳转地址
- 功能：跳转到指定地址执行，地址范围限制在当前PC寄存器所指向的指令地址的±32MB范围

跳转指令也叫程序转移指令。

写汇编程序时，可以跳转到一个绝对地址，如：

B 0x1234 (注： B #0x1234是错误的)

编译器会把该绝对地址转换为相对地址放入指令中。

3.3.1 分支指令

signed_immed_24 间接提供目标地址，真正的目标地址是由处理器根据这个有符号数和当前的PC值计算出来的。

具体计算为：先将 signed_immed_24 左移两位（即具有26位的偏移量），并扩展为32位有符号数，然后再将这32位有符号数与 PC 的当前值相加，得到实际的跳转地址。

因此B 和 BL 指令转移的偏移量为 26 位，即转移的跨度为前后 32MB 地址空间。

BNE forward

ADD R1,R2,#4

forward SUB R1,R2,#56

3.3.1 分支指令

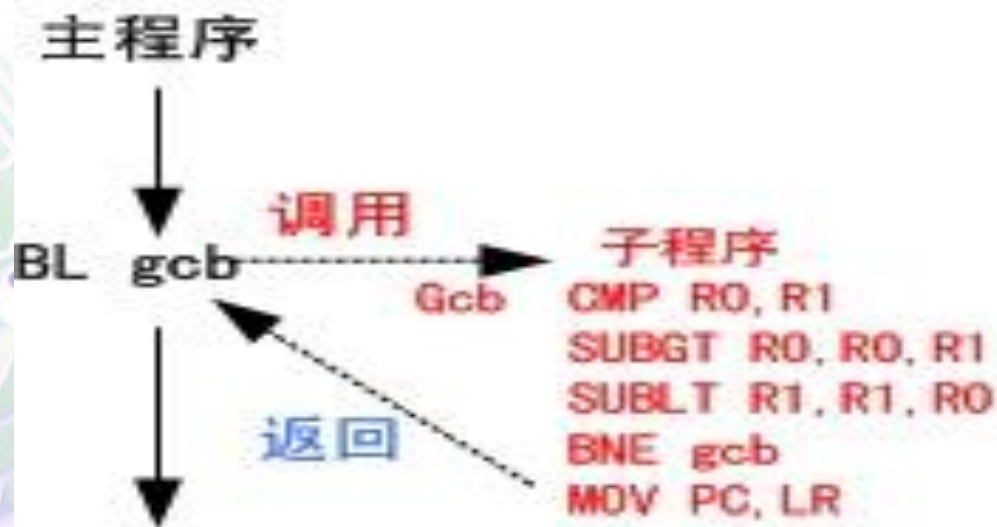
例：现已知寄存器 R0 中存放了数据 a，寄存器 R1 中存放了数据 b，编写一个程序段，求取 a 和 b 的**最大公约数**并将其存入寄存器 R0。（《九章算术》的更相减损法）

```
gcb  CMP R0,R1      ; 比较a和b的大小
      SUBGT R0,R0,R1 ; 如果a>b,则a=a-b
      SUBLT R1,R1,R0 ; 如果a<b,则b=b-a
      BNE gcb        ; 如果a!=b,则返回gcb
      MOV PC,LR      ; 如果a=b,则返回主程序
```

3.3.1 分支指令

B和**BL**的区别在于：BL在跳转之前会把BL指令的下一条指令地址（**断点地址**）保存到连接寄存器 LR（R14），因此程序在必要的时候可以通过将 LR 的内容加载到 PC 中，使程序返回到跳转点。

BL 指令经常被用来调用一个子程序。



3.3.1 分支指令

二、带状态切换的跳转指令BX

BX 指令的格式为：BX{<cond>} Rm

- cond表示指令执行条件
- Rm寄存器，值是绝对地址值，不是偏移量，在指令执行后，Rm中的地址值与#0xFFFF FFFE 进行“与”运算，再被复制到程序计数器PC。
- Rm[0]为1时，强制程序从ARM指令状态跳转到Thumb指令状态。
- Rm[0]为0时，强制程序从Thumb 指令状态跳转到ARM指令状态。
- 功能：跳转到指令中所指定的目标地址，并实现状态切换。
(T<-Rm[0])

3.3.1 分支指令

指令编码格式

31	28 27																				4	3	0		
cond		0	0	0	1	0	0	1	0	1	1	1	1	1	1	1	1	1	1	0	0	0	1	Rm	

其中，Rm为目标地址寄存器。

3.3.1 分支指令

- 例如：

	CODE32	;	32位编码
ARM1	LDR R0, =Thumb1	;	把Thumb地址赋给R0, 末位自动置1
	MOV LR, PC	;	设置返回地址
	BX R0	;	跳转, 切换状态
	ADD R1, R2, #2		
	...		
	CODE16	;	16位编码
Thumb1	ADD R1, R3, #1	;	Thumb程序
	...		
	BX LR	;	跳转到返回地址, 状态切换

3.3.1 分支指令

三、带连接和状态切换的连接跳转指令 BLX

BLX 指令的格式有两种：

BLX <target> 和 **BLX{<cond>} Rm**

以**target**方式提供目标地址的 BLX 指令的功能是：把程序**跳转到**指令中所指定的**目标地址**继续执行，并同时将处理器的工作状态从ARM状态**切换到**Thumb状态，并将**下一条的地址**保存到**寄存器 LR** 中。

而以 **Rm** 方式提供目标地址的 BLX 指令，除了**跳转**和**下一条的地址**保存到LR之外，也可进行**状态切换**，但其切换的依据是**Rm最低位的值**。如果值为**0**，则目标地址处应为**ARM指令**，如果值为**1**，则目标地址处应为 **Thumb 指令**。

3.3.1 分支指令

例

	CODE32	;	32位编码
ARM1	LDR R0, =Thumb1+1	;	把Thumb地址赋给R0, 末位置1
	MOV LR, PC	;	设置返回地址, 此条指令注销
	BLX R0	;	跳转, 切换状态,
		;	LR ← 下一条指令地址
	ADD R1, R2, #2		
	...		
	CODE16	;	16位编码
Thumb1	ADD R1, R3, #1	;	Thumb程序
	...		
	BX LR	;	跳转到返回地址, 状态切换

注：某些型号的CPU不支持BLX指令，比如S3C2440

3.3.1 分支指令

另一种实现指令跳转的方式是通过直接向 PC 寄存器中写入目标地址值，实现在 4GB 地址空间中任意跳转，这种跳转又称为长跳转。

如果在长跳转指令之前使用 “MOV LR, PC” 等指令，可以保存将来返回的地址值，也就实现了在 4GB 的地址空间中的子程序调用。

3.3.2 数据处理指令

3.3.2.1 数据处理指令概述

1、ARM数据处理指令的功能

主要完成寄存器中数据的**算术**和**逻辑运算**操作。

2、ARM数据处理指令的特点

- 操作数来源：所有的操作数要么来自**寄存器**，要么来自**立即数**，**不会**来自存储器。
- 操作结果：如果有结果，则结果一定是为32位宽、或64位宽（长乘法指令），并且放在一个或两个**寄存器**中，**不会**写入存储器。
- 有第二个操作数（除了乘法指令） Operand2：切记其三种形式：立即数、寄存器、寄存器移位。
- **乘法指令**的操作数：**全部是**寄存器。

3.3.2 数据处理指令

3、ARM数据处理指令分类

22条可分为5类：

- 算术运算指令： ADD ADC
 SUB SBC RSB RSC
 MUL MLA UMULL UMLAL SMULL SMLAL
- 逻辑运算指令： AND ORR EOR BIC
- 数据传送指令： MOV MVN
- 比较指令： CMP CMN
- 测试指令： TST TEQ

上述指令只能对寄存器操作，不能针对存储器。

3.3.2 数据处理指令

4、数据处理指令对程序状态寄存器CPSR的影响

指令中可以选择s后缀，以影响状态标志。但是比较指令（CMP和CMN）和测试指令（TST和TEQ）不需要后缀S，它们总会直接影响CPSR中的状态标志。

关于恢复CPSR原值问题：

如果指令带有S后缀（除了比较指令以外），同时又以PC为目标寄存器进行操作：

- 在异常模式下：则操作的同时从SPSR恢复CPSR。

3.3.2 数据处理指令

- 在异常模式下：则操作的同时从SPSR恢复CPSR。比如：

```
movs pc, #0xff /* cpsr = spsr; pc = 0xff */
```

```
adds pc, r1, #0xff00
```

```
/* cpsr = spsr; pc = r1 + 0xff00 */
```

```
ands pc, r1, r2 /* cpsr = spsr; pc = r1 & r2; */
```

- 在user或者system模式：会产生不可预料的结果，因为在这两种模式下没有SPSR。

5、数据处理指令的详细列表（未含6条乘法指令）

见下页

3.3.2 数据处理指令

表 3.3 数据处理指令说明

助记符	说明	操作	条件码位置
MOV Rd, operand2	数据转送	$Rd \leftarrow \text{operand2}$	MOV {cond} {S}
MVN Rd, operand2	数据非转送	$Rd \leftarrow \neg(\text{operand2})$	MVN {cond} {S}
ADD Rd, Rn, operand2	加法运算指令	$Rd \leftarrow Rn + \text{operand2}$	ADD {cond} {S}
SUB Rd, Rn, operand2	减法运算指令	$Rd \leftarrow Rn - \text{operand2}$	SUB {cond} {S}
RSB Rd, Rn, operand2	逆向减法指令	$Rd \leftarrow \text{operand2} - Rn$	RSB {cond} {S}
ADC Rd, Rn, operand2	带进位加法指令	$Rd \leftarrow Rn + \text{operand2} + \text{Carry}$	ADC {cond} {S}
SBC Rd, Rn, operand2	带进位减法指令	$Rd \leftarrow \text{operand2} - (\neg \text{Carry})$	SBC {cond} {S}
RSC Rd, Rn, operand2	带进位逆向减法指令	$Rd \leftarrow \text{operand2} - Rn - (\neg \text{Carry})$	RSC {cond} {S}
AND Rd, Rn, operand2	逻辑与运算指令	$Rd \leftarrow Rn \text{ AND } \text{operand2}$	AND {cond} {S}
ORR Rd, Rn, operand2	逻辑或运算指令	$Rd \leftarrow Rn \text{ OR } \text{operand2}$	ORR {cond} {S}
EOR Rd, Rn, operand2	逻辑异或运算指令	$Rd \leftarrow Rn \text{ XOR } \text{operand2}$	EOR {cond} {S}
BIC Rd, Rn, operand2	位清除指令	$Rd \leftarrow Rn \& (\neg \text{operand2})$	BIC {cond} {S}
CMP Rd, operand2	比较指令	标志 N、C、Z、V $\leftarrow Rn - \text{operand2}$	CMP {cond}
CMN Rd, operand2	负数比较指令	标志 N、C、Z、V $\leftarrow Rn + \text{operand2}$	CMN {cond}
TST Rd, operand2	位测试指令	标志 N、C、Z、V $\leftarrow Rn \text{ AND } \text{operand2}$	TST {cond}
TEQ Rd, operand2	相等测试指令	标志 N、C、Z、V $\leftarrow Rn \text{ OR } \text{operand2}$	TEQ {cond}

3.3.2 数据处理指令

指令编码格式

31	28	27	26	25	24	21	20	19	16	15	12	11	0
cond		0	0	I	opcode	S	Rn		Rd		operand2		

其中，opcode为数据处理指令操作码。I用于区别立即数(I为1)或寄存器移位(I为0)。S用于设置标志位。Rn为第一操作数寄存器，Rd为目标寄存器。operand2为第二个操作数，若指令不需要全部的可用操作数时(如MOV指令的Rn)，不用的寄存器域应设置为0(由编译器自动完成)。对于比较指令，b20位固定为1。

3.3.2 数据处理指令

3.3.2.2 算术运算指令

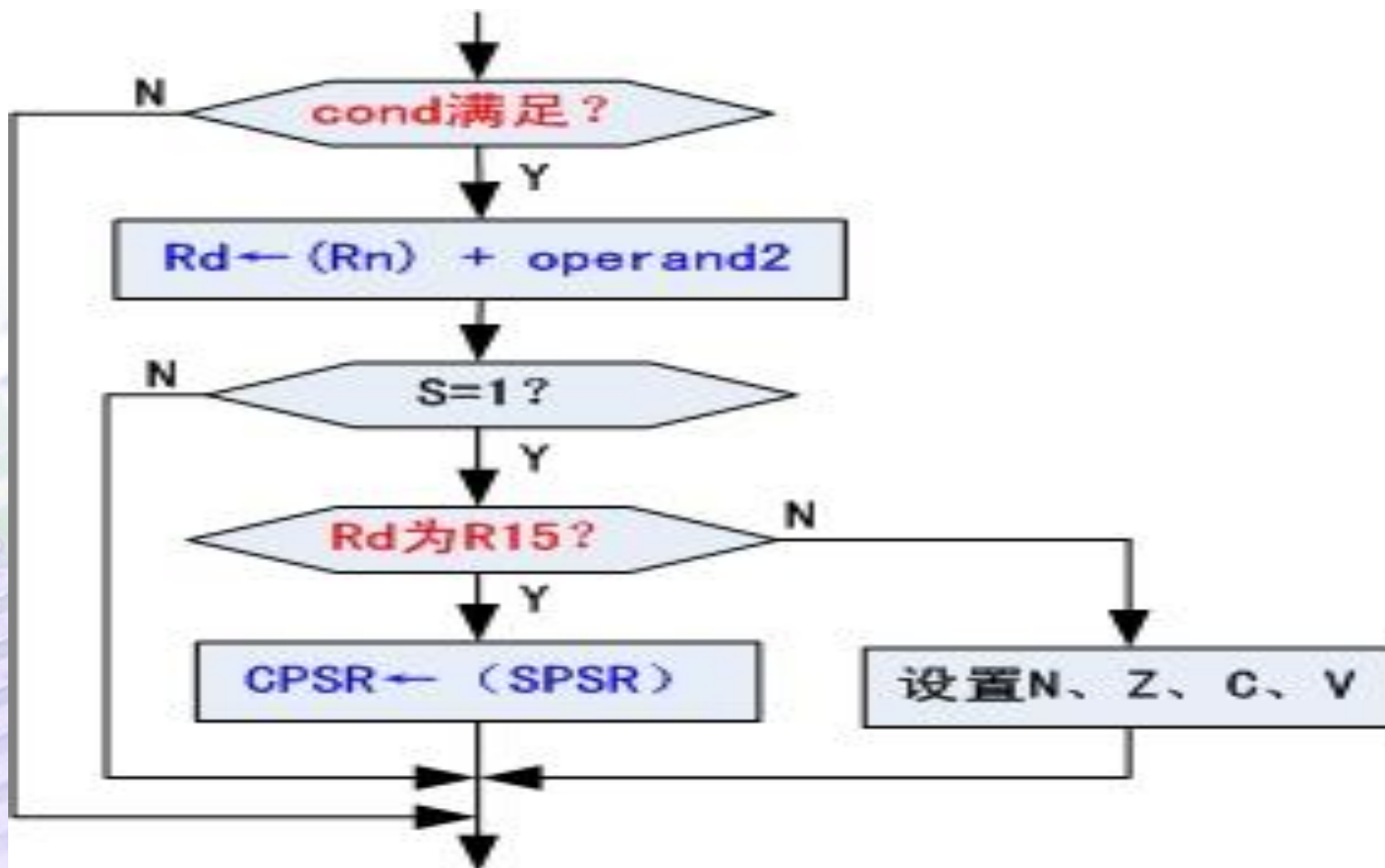
一、加减运算指令

1、ADD——加法运算指令

- 指令格式：ADD{cond}{S} Rd, Rn, operand2
- ADD指令把第1源操作数寄存器Rn和第2源操作数operand2相加后，将结果存放到目的寄存器 Rd。

3.3.2 数据处理指令

ADD指令的执行如下：



3.3.2 数据处理指令

指令举例

改为0X256行吗？

ADD R0, R1, R2 ; $R0 \leftarrow (R1) + (R2)$

ADD R0, R1, #255 ; $R0 \leftarrow (R1) + 255$

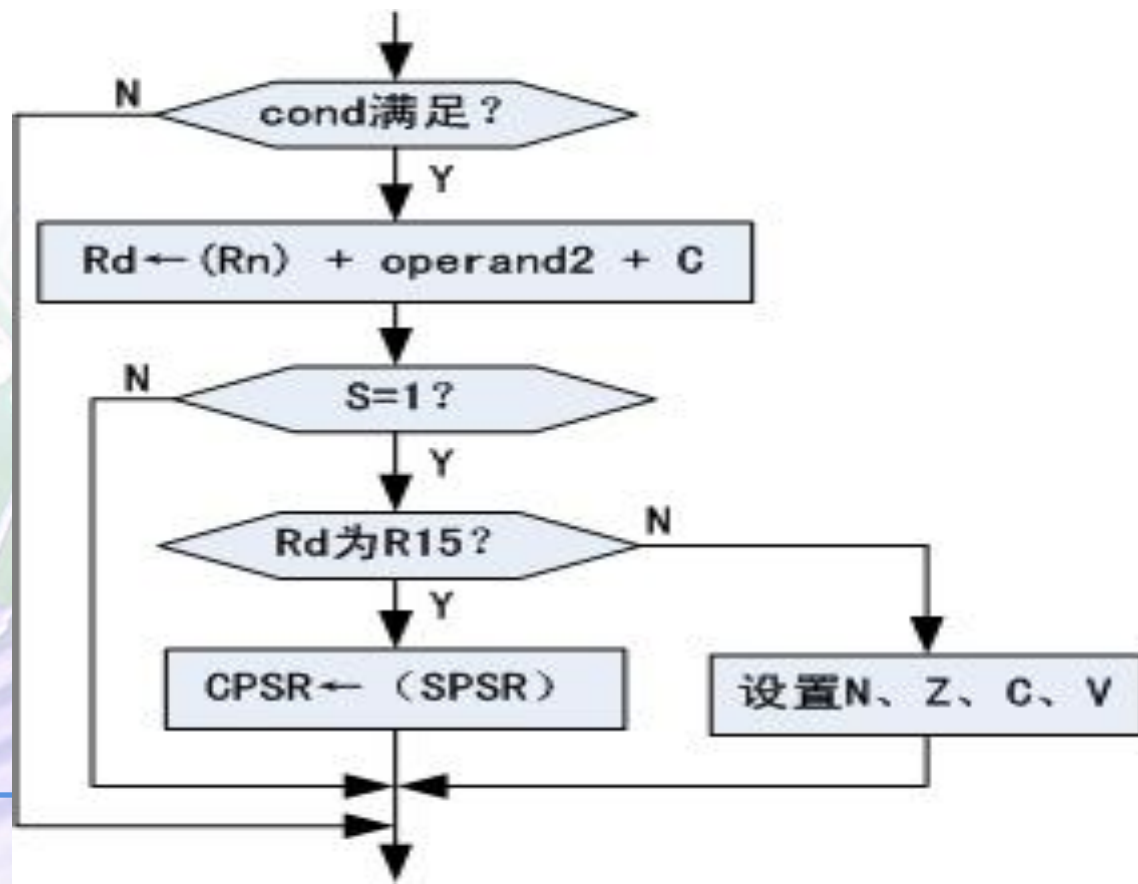
ADD R0, R2, R3, LSL#1 ; $R0 \leftarrow (R2) + (R3 \ll 1)$

受影响的CPSR标志位	取值
N	寄存器Rd[31]被复制到N
Z	如果Rd为0，则Z=1，否则Z=0
C	运算结果有进位C=1，否则C=0
V	运算结果有溢出V=1，否则V=0

3.3.2 数据处理指令

2、ADC——带进位加法指令

- 指令格式：ADC{cond}{S} Rd, Rn, operand2
- ADC指令将operand2的数据与Rn的值相加，再加上CPSR中的C条件标志位，结果保存到Rd寄存器。



3.3.2 数据处理指令

标志位只修改 N、Z、C、V这4个标志位。

ADC通常用来实现字长大于32位的加法运算。

例：有 两个128位数，第一个数由高到低存放在寄存器 R7~R4 中，第二个数由高到低存放在寄存器 R11~R8 中，请编写程序把两个数相加，运算结果由高到低存放到寄存器 R3~R0中。

能否不要这些 S ？

ADDS R0, R4, R8 ; 加低位字，不带进位

ADCS R1, R5, R9 ; 加第二个字，带进位

ADCS R2, R6, R10 ; 加第三个字，带进位

ADCS R3, R7, R11 ; 加第四个字，带进位

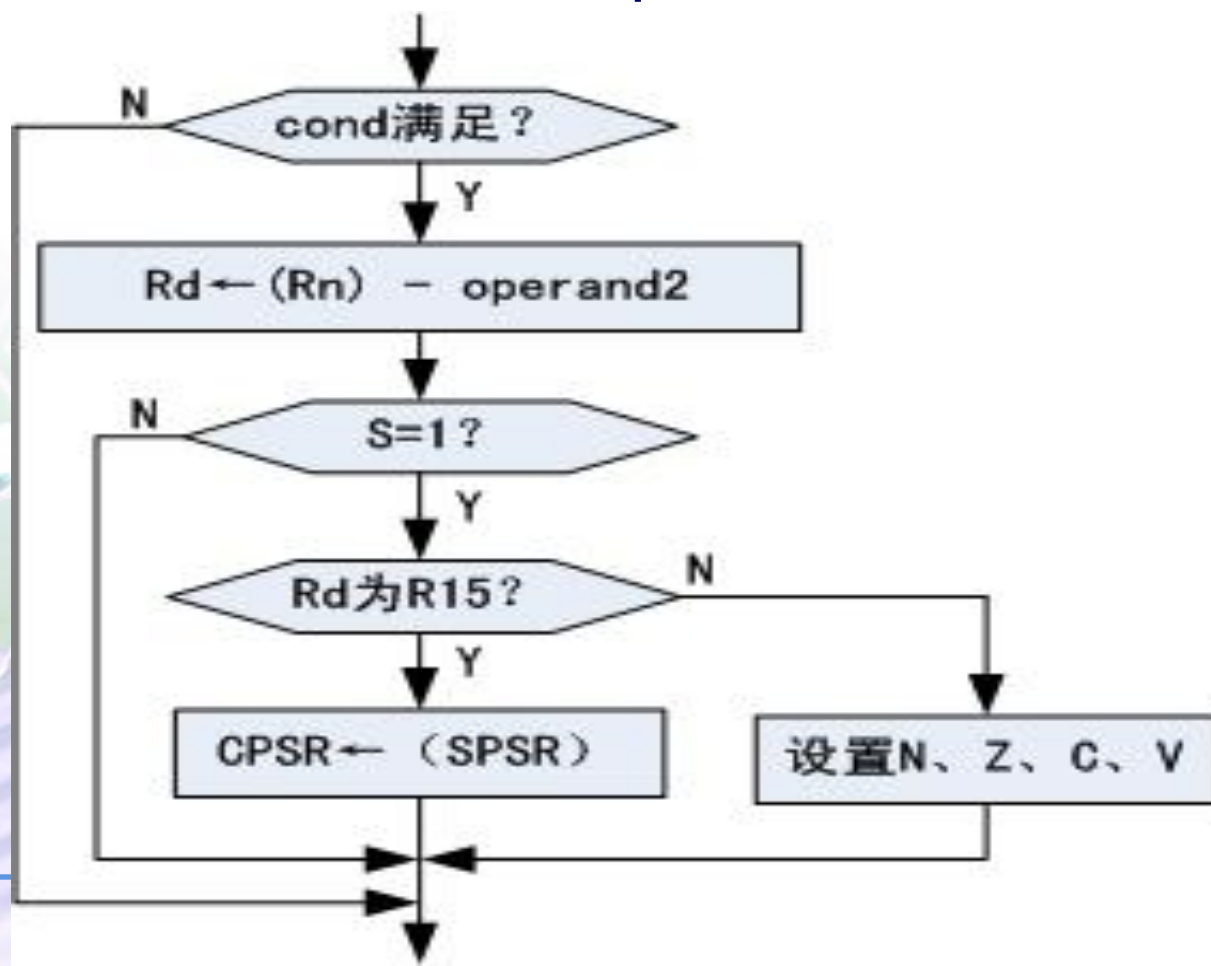
这段程序
有缺陷吗？

3.3.2 数据处理指令

3、SUB——减法运算指令

指令格式：SUB{cond}{S} Rd, Rn, operand2

SUB指令用寄存器Rn减去operand2，结果保存到Rd中。



3.3.2 数据处理指令

例:

SUB R0, R1, R2 ; $R0 \leftarrow (R1) - (R2)$

SUB R0, R1, #256 ; $R0 \leftarrow (R1) - 256$

SUB R0, R2, R3, LSL#1; $R0 \leftarrow (R2) - (R3 \ll 1)$

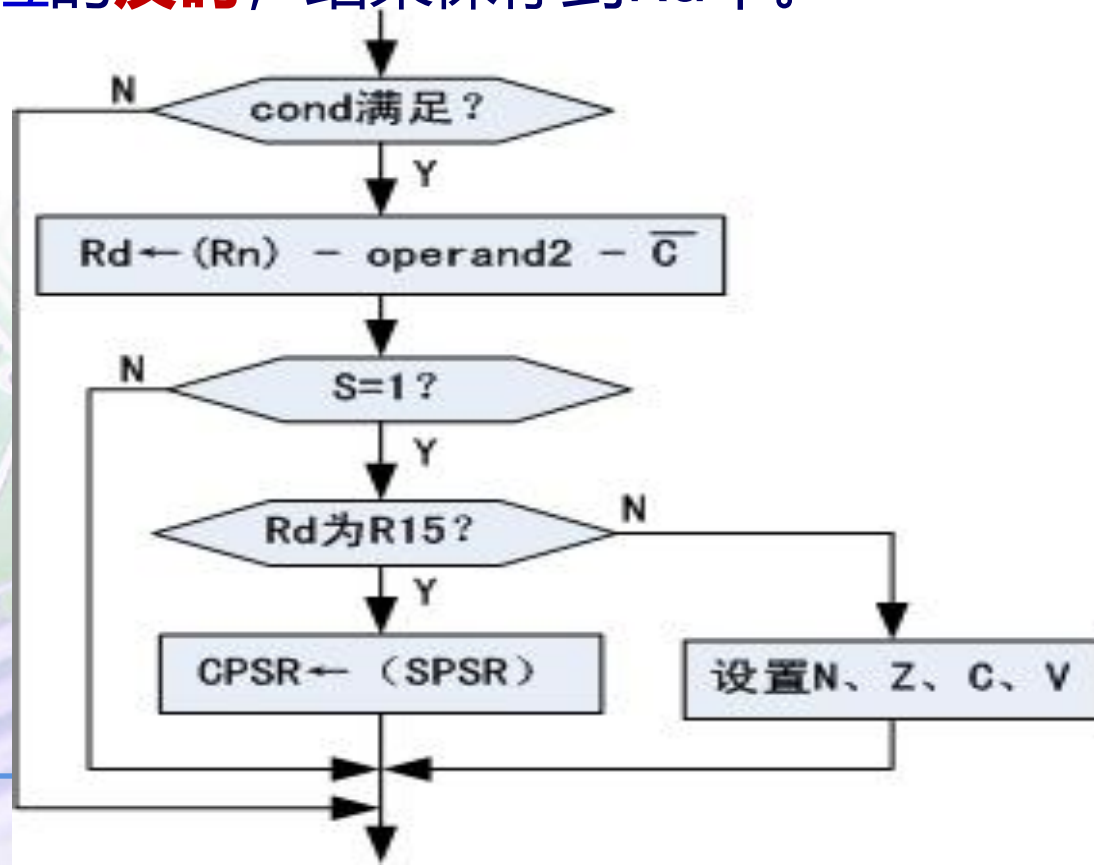
受影响的CPSR标志位	取值
N	寄存器Rd[31]被复制到N
Z	如果Rd为0则Z=1, 否则Z=0
<u>C</u>	运算结果有借位则C=0, 否则C=1
V	运算结果有溢出则V=1, 否则V=0

3.3.2 数据处理指令

4、SBC——带进位减法指令

指令格式：SBC{cond}{S} Rd, Rn, operand2

SBC指令用寄存器Rn减去operand2，再减去CPSR中的C条件标志位的**反码**，结果保存到Rd中。



3.3.2 数据处理指令

标志位的修改同 SUB。

该指令主要用于字长大于 32 位的数据的减法运算。

指令举例：使用SBC实现64位减法

```
SUBS  R4, R0, R2
```

```
SBC   R5, R1, R3   ; (R5,R4) = (R1,R0)-(R3,R2)
```

存放在R0和R1中的64位数(R0是低位) 减去存放于R2和R3中的64位数(R2是低位) ， 结果存放到R4和R5两个寄存器中(R4是低位) 。

3.3.2 数据处理指令

5、RSB——逆(反)向减法指令

指令格式：RSB{cond}{S} Rd, Rn, operand2

RSB指令用operand2减去寄存器Rn，结果保存到Rd中。

指令举例如下：

RSB R3, R1, #0xFF00 ; $R3 = 0xFF00 - R1$

RSB R0, R1, #256 ; $R0 \leftarrow 256 - (R1)$

RSBS R1, R2, R2, LSL #2
; $R1 = R2 \ll 2 - R2$
; $(R1 = R2 \times 3)$

3.3.2 数据处理指令

6、RSC——带进位反向减法指令

指令格式：RSC{cond}{S} Rd, Rn, operand2

RSC 指令用寄存器operand2减去Rn，再减去CPSR中的C条件标志位的反码，结果保存到Rd中。

指令举例：求64位数值的负数

RSBS R2, R0, #0 ; 求一个32位数的负数

RSC R3, R1, #0 ; 使用RSC指令实现

3.3.2 数据处理指令

二、乘法指令

ARM有两类乘法指令：

- 32位的乘法指令，即乘法操作的结果为32位；
- 64位的乘法指令，即乘法操作的结果为64位。

助记符	说明	意义	效果
MUL Rd, Rm, Rs	32 位乘法指令	$Rd \leftarrow Rm * Rs$ ($Rd \neq Rm$)	MUL {cond} {S}
MLA Rd, Rm, Rs	32 位乘加指令	$Rd \leftarrow Rm * Rs + Rd$ ($Rd \neq Rm$)	MLA {cond} {S}
UMULL RdLo, RdHi, Rm, Rs	64 位无符号乘法	$(RdLo, RdHi) \leftarrow Rm * Rs$	UMULL {cond} {S}
UMLAL RdLo, RdHi, Rm, Rs	64 位无符号乘加	$(RdLo, RdHi) \leftarrow Rm * Rs + (RdLo, RdHi)$	UMULL {cond} {S}
SMULL RdLo, RdHi, Rm, Rs	64 位有符号乘法	$(RdLo, RdHi) \leftarrow Rm * Rs$	SMULL {cond} {S}
SMLAL RdLo, RdHi, Rm, Rs	64 位有符号乘加	$(RdLo, RdHi) \leftarrow Rm * Rs + (RdLo, RdHi)$	SMULL {cond} {S}

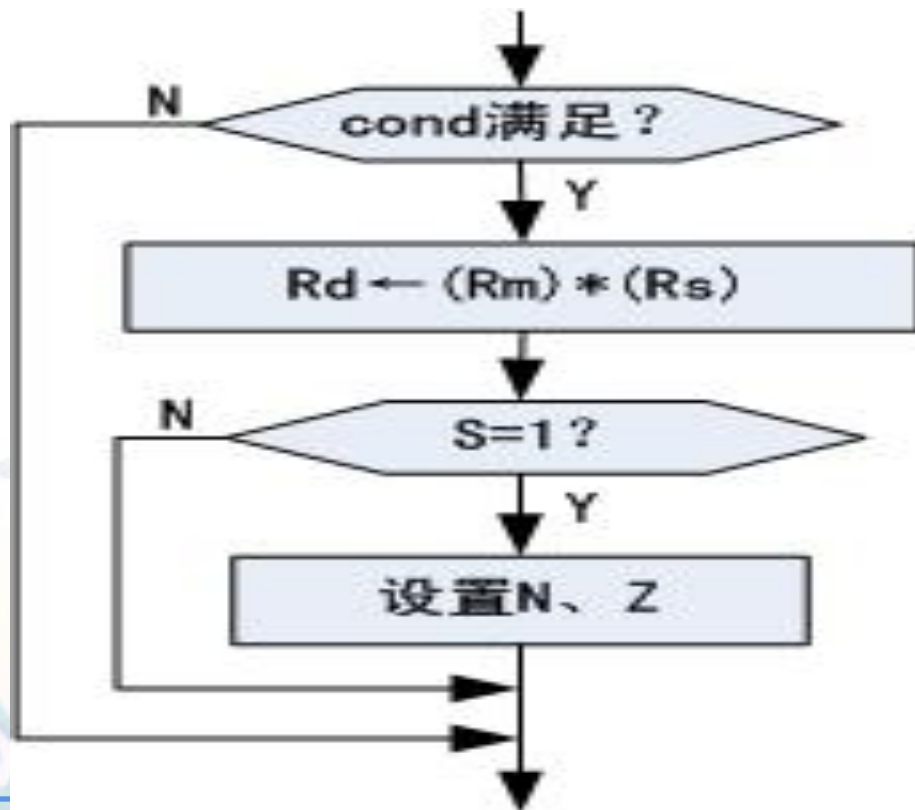
3.3.2 数据处理指令

1、 MUL——32位乘法指令

指令格式: MUL{cond}{S} Rd, Rm, Rs ; $Rd \leftarrow Rm * Rs$

MUL指令将Rm和Rs中的值相乘, 结果的低32位保存到Rd中。

注: $Rd \neq Rm$



3.3.2 数据处理指令

受影响的CPSR标志位	取值
N	寄存器Rd[31]被复制到N
Z	如果Rd为0则Z=1，否则Z=0

指令举例如下：

MUL R1, R2, R3 ; R1=R2×R3

MULS R0, R3, R7 ; R0=R3×R7,

; 设置CPSR的N位和Z位

3.3.2 数据处理指令

2、 MLA——32位乘加指令

格式 $\text{MLA}\{\text{cond}\}\{\text{S}\} \quad \text{Rd}, \text{Rm}, \text{Rs}, \text{Rn} \quad ; \text{Rd} \leftarrow \text{Rm} * \text{Rs} + \text{Rn}$

指令将Rm和Rs中的值相乘，再将乘积加上第3个操作数，结果的低32位保存到Rd中。注： $\text{Rd} \neq \text{Rm}$

标志位的修改同 MUL。

例：

$\text{MLA} \text{ R0}, \text{R1}, \text{R2}, \text{R3} \quad ; \text{R0} \leftarrow (\text{R1}) \times (\text{R2}) + (\text{R3})$

$\text{MLAS} \text{ R0}, \text{R1}, \text{R2}, \text{R3} \quad ; \text{R0} \leftarrow (\text{R1}) \times (\text{R2}) + (\text{R3}),$
；并更新CPSR标志位

3.3.2 数据处理指令

3、 UMULL—64位无符号乘法指令

指令格式：UMULL{cond}{S} RdLo, RdHi, Rm, Rs
; RdHi, RdLo $\leftarrow$ Rm * Rs

UMULL指令将Rm和Rs中的值作无符号数相乘，结果的低32位保存到RdLo中，高32位保存到RdHi中

例：UMULL R0, R1, R5, R8 ; (R1,R0) $\leftarrow$ R5 $\times$ R8

受影响的CPSR标志位	取值
N	寄存器RdHi [31]被复制到N
Z	如果RdHi且RdLo为0，则Z=1，否则Z=0

3.3.2 数据处理指令

4、UMLAL—64位无符号乘加指令

指令格式：UMLAL{cond}{S} RdLo, RdHi, Rm, Rs
; RdHi, RdLo $\leftarrow$ Rm * Rs + RdHi, RdLo

UMLAL指令将Rm和Rs中的值作无符号数相乘，64位乘积与RdHi、RdLo相加，结果的低32位保存到RdLo中，而高32位保存到RdHi中。

标志的修改同 UMULL。

指令举例如下：

UMLAL R0, R1, R5, R8 ; (R1,R0) $\leftarrow$ R5 $\times$ R8 + (R1,R0)

3.3.2 数据处理指令

5、SMULL—64位有符号乘法指令

指令格式: SMULL{cond}{S} RdLo, RdHi, Rm, Rs
; RdHi, RdLo $\leftarrow$ Rm * Rs

SMULL指令将Rm和Rs中的值作有符号数相乘, 结果的低32位保存到RdLo中, 而高32位保存到RdHi中。

例如: SMULL R2, R3, R7, R6 ; (R3,R2) $\leftarrow$ R7 $\times$ R6

受影响的CPSR标志位	取值
N	寄存器RdHi [31]被复制到N
Z	如果RdHi且RdLo为0, 则Z=1, 否则Z=0

3.3.2 数据处理指令

6、SMLAL—64位有符号乘加指令

指令格式: SMLAL{cond}{S} RdLo, RdHi, Rm, Rs
; RdHi, RdLo $\leftarrow$ Rm * Rs + RdHi, RdLo

SMLAL指令将Rm和Rs中的值作有符号数相乘, 64位乘积与RdHi、RdLo相加, 结果的低32位保存到RdLo中, 高32位保存到RdHi中。

标志的修改同 SMULL。

指令举例如下: SMLAL R2, R3, R7, R6
; (R3,R2) $\leftarrow$ R7 $\times$ R6 + (R3,R2)

3.3.2 数据处理指令

7、乘法指令的特点

- ①不支持第2操作数为立即数。
- ②结果寄存器不能与第一源寄存器相同。
 - Rd、RdHi、RdLo不能与Rm为同一寄存器。
 - RdHi和RdLo不能为同一寄存器。
- ③避免将R15定义为任一操作数或结果寄存器。
- ④早期的ARM处理器仅支持32位乘法指令。ARM7版本和后续的在名字中有M的处理器才支持64位乘法指令。

3.3.2 数据处理指令

⑤ 对标志位的影响

- 对N标志位：若结果是32位指令形式，Rd的第31位是标志位N；对于产生长结果的指令形式，RdHi的第31位是标志位。
- 对Z标志位：如果Rd或RdHi、RdLo为0，则标志位Z置位。
- 对V标志位：乘法指令不影响V标志位。
- 对C标志位：ARM v5及以上的版本不影响C标志位；ARM v5以前的版本，C标志位数值不确定。

3.3.2 数据处理指令

指令编码格式

31	28	27	24	23	22	21	20	19	16	15	12	11	8	7	6	5	4	3	0			
cond		0	0	0	0	opcode		S	Rd/RdHi			Rn/RdLo			Rs		1	0	0	1	Rm	

其中，opcode为乘法指令操作码。S为设置标志位。Rm为被乘数寄存器，Rs为乘数的寄存器。Rn/RdLo用于MLA指令相加的寄存器或64位乘法指令的目标寄存器(低32位)。Rd/RdHi用于目标寄存器或64位乘法指令的目标寄存器(高32位)。若指令不需要全部的可用操作数时(如MUL指令的Rn)，不用的寄存器域应设置为0(由编译器自动完成)。

3.3.2 数据处理指令

3.3.2.3 逻辑运算指令（按位逻辑操作指令）

按位逻辑操作指令包括：逻辑与操作指令AND、逻辑或操作指令ORR、逻辑异或操作指令EOR 以及 位清除指令BIC。

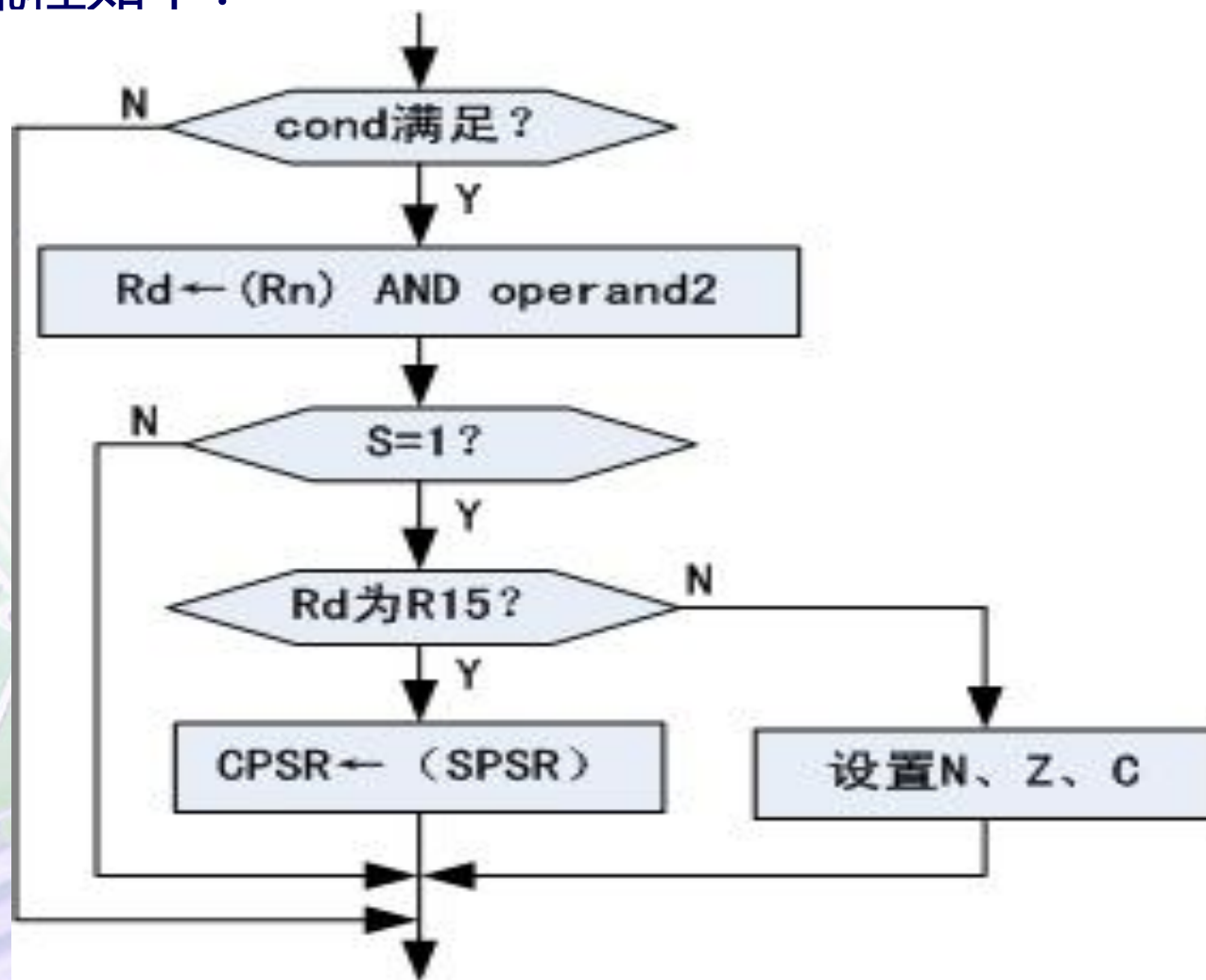
1、AND——逻辑 “与” 操作指令

指令格式：AND{cond}{S} Rd, Rn, operand2

AND指令将operand2的值与寄存器Rn的值按位逻辑 “与” 操作，结果保存到Rd中。

3.3.2 数据处理指令

指令流程如下：



3.3.2 数据处理指令

受影响的CPSR标志位	取值
N	寄存器Rd[31]被复制到N
Z	如果Rd为0则Z=1，否则Z=0
C	不影响C标志位

指令举例如下：

ANDS R0, R0, #0x01 ; R0 = R0&0x01
; 取出最低位数据

AND R2, R1, R3 ; R2 = R1&R3

AND指令可用于提取寄存器中某些位的值，也可以用于把指定位清0。

3.3.2 数据处理指令

2、ORR——逻辑“或”操作指令

指令格式： ORR{cond}{S} Rd, Rn, operand2

ORR指令将operand2的值与寄存器Rn的值按位逻辑“或”操作，结果保存到Rd中。

指令举例如下：

ORR R0, R0, #0x0F ; 将R0的低4位置1

ORR指令用于将寄存器中某些位的值设置成1。

标志位的影响同AND。

3.3.2 数据处理指令

例：请把寄存器R2中的高8位数据传送到寄存器R3的低8位，原R3低24置换为高24位。

注意：R2原来的内容没有变

MOV R0, R2, LSR #24

；将R2右移24位，即将其高8位移至低8位送R0。

ORR R3, R0, R3, LSL #8

；将R3左移8位后将R0低8位送至R3。

R3原来的高24位内容变了

3.3.2 数据处理指令

3、EOR——逻辑“异或”操作指令

指令格式：EOR{cond}{S} Rd, Rn, operand2

EOR指令将operand2的值与寄存器Rn的值按位逻辑“异或”操作，结果保存到Rd中。

指令举例如下：

EOR R1, R1, #0x0F ; 将R1的低4位取反

EORS R0, R5, #0x01 ; R0<- R5异或0x01,
; 并影响标志位

EOR指令可用于将寄存器中某些位的值取反。将某一位与0异或，该位值**不变**；与1异或，该位值被**求反**。

标志位的影响同AND。

3.3.2 数据处理指令

4、BIC——位清除指令

指令格式：BIC{cond}{S} Rd, Rn, operand2

BIC指令将寄存器Rn的值与operand2的值的反码按位逻辑“与”操作，结果保存到Rd中。

指令举例如下：

BIC R1, R1, #0x0F ; 将R1的低4位清0,
; 其它位不变

BIC指令可用于将寄存器中某些位的值清除为0。将某一位与1做BIC操作，该位值被清除为0；将某一位与0做BIC操作，该位值不变。

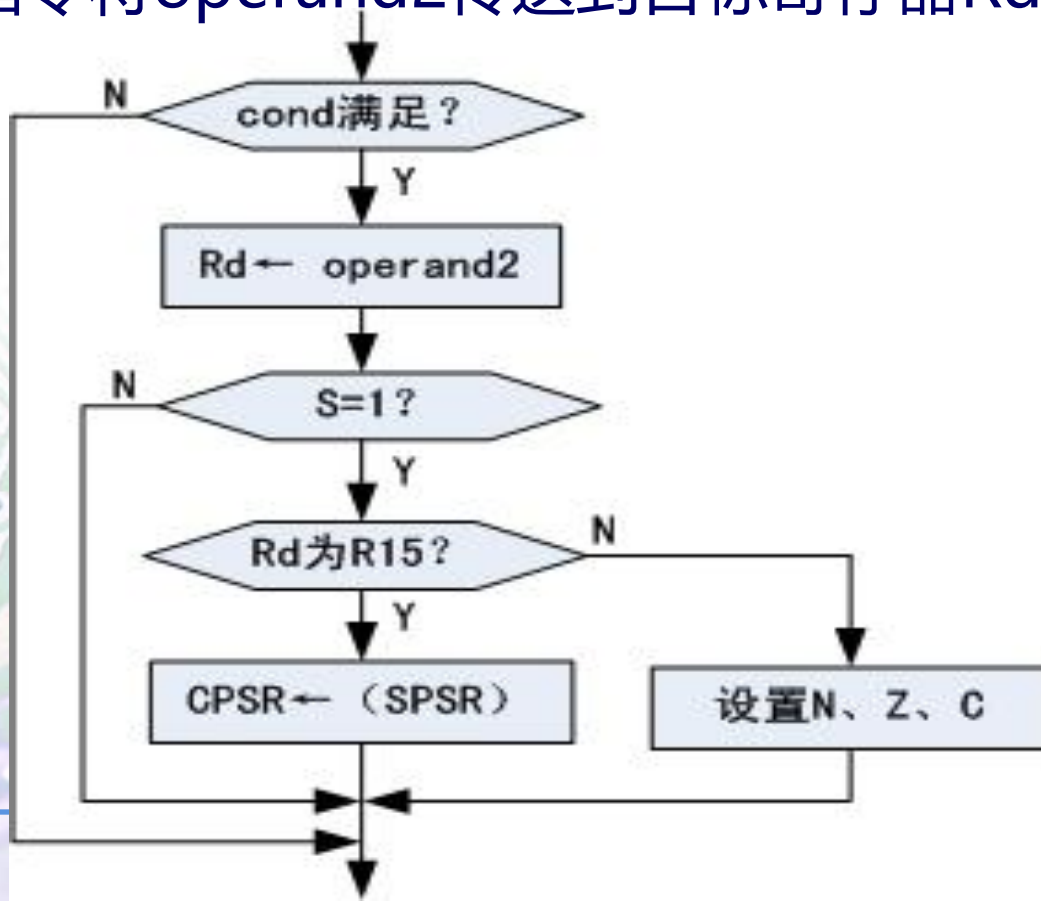
3.3.2 数据处理指令

3.3.2.4 数据传送指令

1、MOV——数据传送指令

指令格式：MOV{cond}{S} Rd, operand2

MOV指令将operand2传送到目标寄存器Rd中。



3.3.2 数据处理指令

受影响的CPSR标志位	取值
N	寄存器Rd[31]被复制到N
Z	如果Rd为0，则Z=1，否则Z=0
C	C=0

指令举例如下：

MOV R2, #0x7E ; 将立即数0x7E传送到寄存器R2中

MOVS R1, R0, LSL#3

; 将寄存器R0 * 8传送到寄存器R1，并影响标志位

MOV PC, LR ; PC←LR, 子程序返回

MOVS PC, LR ; PC←LR, 异常模式下返回

; CPSR←SPSR

3.3.2 数据处理指令

MOV指令的功能总结

- 寄存器之间传送。
- 立即数传送到寄存器中。（8位立即数位图）
- 实现单纯的移位操作。MOV Rd, Rd, LSL, #3
- 实现子程序调用、从子程序中返回。当PC寄存器作为目标寄存器时可以实现程序跳转。
- 实现异常模式的返回，并把当前处理器模式的SPSR寄存器内容复制到CPSR中。

3.3.2 数据处理指令

2、MVN——数据求反传送指令

指令格式：MVN{cond}{S} Rd, operand2

MVN指令将operand2按位取反后传送到目标寄存器Rd中。

指令举例如下：

MVN R1, #0xFF ; R1←0xFFFFFFFF00

MVN R1, R2 ; R1← R2取反

3.3.2 数据处理指令

3.3.2.5 比较指令

1、CMP——比较指令

指令格式：CMP{cond} Rn, operand2

CMP指令将寄存器Rn的值减去operand2的值，但不存储运算结果，只根据操作的结果更新CPSR中的相应条件标志位，以便后面的指令根据相应的条件标志来判断是否执行。

受影响的CPSR标志位	取值
N	运算结果的第31位被复制到N
Z	运算结果为0则Z=1，否则Z=0
C	运算结果有借位则C=0，否则C=1
V	运算结果有溢出则V=1，否则V=0

3.3.2 数据处理指令

指令举例如下：

CMP R1, R0

； (R1)-(R0)； 根据结果设置CPSR的标志位。

CMP R1, #100

； (R1)-100； 根据结果设置CPSR的标志位。

注意：比较类指令本身带有更新 CPSR的功能，故在该指令中不能使用后缀 S。

CMP指令与SUBS指令的区别？

3.3.2 数据处理指令

2、CMN——负数比较指令

指令格式：CMN{cond} Rn, operand2

CMN指令将寄存器Rn的值减去operand2的负数（即加上operand2的值），但不存储运算结果，只根据操作的结果更新CPSR中的相应条件标志位，以便后面的指令根据相应的条件标志来判断是否执行。

指令举例如下：

CMN R0, #1 ; R0+1, 判断R0是否为-1。

3.3.2 数据处理指令

3.3.2.6 测试指令

1、TST——位测试指令

指令格式：TST{cond} Rn, operand2

TST指令将寄存器Rn的值与operand2的值按位逻辑“与”操作，但不存储运算结果，只根据操作的结果更新CPSR中的相应条件标志位。该指令一般用来检测是否设置了特定的位。

指令举例如下：

TST R0, #0x01 ; 判断R0的最低位是否为0

TST R1, #0x0F ; 判断R1的低4位是否为0

3.3.2 数据处理指令

2、TEQ——测试相等指令

指令格式：TEQ{cond} Rn, operand2

TEQ指令将寄存器Rn的值与operand2的值按位逻辑“异或”操作，但不存储运算结果，只根据操作的结果更新CPSR中的相应条件标志位，以便后面的指令根据相应的条件标志来判断是否执行。

指令举例如下：

TEQ R0, R1 ; 比较R0与R1是否相等
; (不影响V位和C位)

3.3.3 存储器访问指令

ARM微处理器用加载/存储指令访问存储器，实现在寄存器和存储器之间传送数据。

由于ARM处理器对外设寄存器、I/O映射空间与存储器统一编址，因此，对外围设备的I/O操作也用此类指令。

基本的加载/存储指令仅有5条，分为3种：

- LDR和STR，单寄存器加载/存储指令
- LDM和STM，多寄存器加载/存储指令
- SWP，寄存器和存储器数据交换指令

3.3.3 存储器访问指令

3.3.3.1 单寄存器的存取指令

单寄存器加载/存储指令是ARM在寄存器和存储器间传送**单个字节**和**字**的最灵活方式。

根据传送数据的**类型**不同，单个寄存器存取指令又可以分为以下两类：

- **单字**和**无符号字节**的加载/存储指令
- **半字**和**有符号字节**的加载/存储指令

3.3.3 存储器访问指令

一、单字和无符号字节的加载 / 存储指令

- LDR: 指令从内存中取32位字或8位无符号字节数据放入寄存器;
- STR: 指令将寄存器中的32位字或8位无符号字节数据保存到存储器中。

注意：无符号字节加载时，用0将8位的操作数扩展到32位。

3.3.3 存储器访问指令

(1) 指令格式

- LDR{cond}{T} Rd, <地址>
; 加载指定地址上的字数据, 放入Rd中。
- STR{cond}{T} Rd, <地址>
; 存储Rd中字数据, 到指定地址的存储单元。
- LDR{cond}B{T} Rd, <地址>
; 加载字节数据到Rd低8位数据位中, 高24位为0。
- STR{cond}B{T} Rd, <地址>
; 存储Rd中字节数据, Rd中最低字节为传送数据。

3.3.3 存储器访问指令

T后缀

- T为可选后缀，若指令有T，存储系统将访问看成是处理器是在用户模式下。
- 用于存储器保护。
- 不能与后变址模式、自动变址模式一起使用（即不能改变基址寄存器值）。
- T在用户模式下无效。

3.3.3 存储器访问指令

(2) 操作数寻址方式

LDR/STR指令为基址变址寻址（或寄存器间接寻址），由两部分组成：

- 基地址部分：为一个基址寄存器，可以为任一个通用寄存器；
- 偏移地址部分：这一部分非常灵活，实际就类似第二个操作数，可以有以下3种格式：
 - ✓ 立即数
 - ✓ 寄存器
 - ✓ 寄存器及移位常数。

3.3.3 存储器访问指令

① 立即数

12位立即数是一个无符号的数值。这个数据可以加到基址寄存器，也可以从基址寄存器中减去这个数值。

指令举例如下：

```
LDR    R1, [R0, #0x10]
```

； 将R0+0x10地址处的数据读出，

； 保存到R1中(R0的值不变)

```
LDR    R1, [R0, # -0x10]
```

； 将R0-0x10地址处的数据读出，

； 保存到R1中(R0的值不变)

3.3.3 存储器访问指令

② 寄存器

寄存器中的数值(无符号数)可以加到基址寄存器，也可以从基址寄存器中减去这个数值。

指令举例如下：

- LDR R1, [R0, R2]
; 将R0+R2地址处的数据读出，保存到R1中
- LDR R1, [R0, -R2]
; 将R0-R2地址处的数据读出，保存到R1中

3.3.3 存储器访问指令

③ 寄存器及移位常数

寄存器移位后的值(无符号数)可以加到基址寄存器，也可以从基址寄存器中减去这个数值。

指令举例如下：

```
LDR    R1, [R0, R2, LSL #2]
```

；将 $R0+R2 \times 4$ 地址处的数据读出，

；保存到R1中(R0、R2的值不变)

```
LDR    R1, [R0, -R2, LSL #2]
```

；将 $R0-R2 \times 4$ 地址处的数据读出，

；保存到R1中(R0、R2的值不变)

注意：移位位数只能是5位的立即数，不能使用寄存器指定移位位数。

3.3.3 存储器访问指令

④ PC（即R15）使用的几个问题

- 使用PC作为基址时，使用的数值是当前执行指令的地址加8个字节（取指与执行相差8个字节）。
- PC不能用做偏移寄存器，也不能用于任何变址寻址模式。
- 把一个字加载到PC，将使程序转移到所加载的地址，这是一个公认的实现跳转的方法。但是应当避免将一个字节加载到PC。
- 把PC存到存储器的操作在不同体系结构的处理器中产生不同的结果，应尽可能避免。

3.3.3 存储器访问指令

指令编码格式

31	28	27	26	25	24	23	22	21	20	19	16	15	12	11	0
cond	0	1	I	P	U	B	W	L	Rn	Rd	addressing_mode_specific				

其中，I、P、U、W用于区别不同的地址模式(偏移量)。I为0时，偏移量为12位立即数；I为1时，偏移量为寄存器移位；P表示前/后变址，U表示加/减，W表示回写(! 后缀)。L用于区别加载(L为1时)或存储(L为0时)。B用于区别字节访问(B为1时)或字访问(B为0时)。Rn为基址寄存器，Rd为源/目标寄存器。

3.3.3 存储器访问指令

二、半字和有符号字节的加载 / 存储指令

这类LDR/STR指令可实现**半字**（有符号和无符号）、**有符号字节**数据的传送。

特点：

- 偏移量格式、寻址方式与加载 / 存储字和无符号字节指令基本相同。
- 立即数偏移量**限定在8位**，寄存器偏移量**不可经过移位得到**。

3.3.3 存储器访问指令

(1) 指令格式如下

- LDR {cond}**SB** Rd, <地址>; 加载指定地址上**有符号字节**到Rd中, 高24位**用符号位扩展**
- LDR {cond}**SH** Rd, <地址>; 加载指定地址上的**有符号半字**到Rd中, 高16位**用符号位扩展**。
- LDR {cond}**H** Rd, <地址>; 加载**无符号半字数据**到Rd的低16位,高16位**清零**。
- STR{cond}**H** Rd, <地址>; 存储Rd中的低16位**半字数据**。

存储有符号数据和无符号数据之间**没有差别**。

3.3.3 存储器访问指令

(2) 两点说明:

符号位——有符号字节或有符号半字的加载，用“符号位”扩展到32位；无符号半字传送是用0扩展到32位。

地址对齐——对半字传送的地址必须为偶数。非半字对齐的半字加载将使Rd内容不可靠；非半字对齐的半字存储将使指定地址的2字节存储内容不可靠。

3.3.3 存储器访问指令

指令编码格式

31	28	27	26	25	24	23	22	21	20	19	16	15	12	11	8	7	6	5	4	3	0										
cond				0	0	0	P	U	I	W	L	Rn				Rd				addr_mode				1	S	H	1	addr_mode			

其中，I、P、U、W用于区别不同的地址模式(偏移量)。当I为0时，偏移量为8位立即数；当I为1时，偏移量为寄存器偏移。这里P表示前/后变址，U表示加/减，W表示回写。L用于区别加载(L为1)或存储(L为0)。S用于区别有符号访问(S为1)和无符号访问(S为0)。H用于区别半字访问(H为1)或字节访问(H为0)。Rn为基址寄存器。Rd为源/目标寄存器。

3.3.3 存储器访问指令

(3) 指令举例

例：已知 $(R0) = 0X00000000$ 和 $(R1) = 0X00009000$ ，并已知在存储器中首地址为 $0X00009000$ 的区域中存放了数据 $0X01010101$ ，在首地址为 $0X00009004$ 的区域存放了数据 $0X02020202$ 。试写出执行了指令 `LDR R0, [R1, #4]` 后 $R0$ 和 $R1$ 中的数据。

$(R0) = 0X02020202$ $(R1) = 0X00009000$

例：条件如上题，试写出执行了指令 `LDR R0, [R1, #4]!` 后的 $R0$ 和 $R1$ 中的数据。

$(R0) = 0X02020202$ $(R1) = 0X00009004$

3.3.3 存储器访问指令

例：条件如上题，试写出执行了指令 `LDR R0, [R1], #4` 后 R0 和 R1 中的数据。

`(R0) = 0X01010101`

`(R1) = 0X00009004`

`LDR R1, [R0, #0X10]`

;将R0+16地址处的数据读出，保存到R1中（R0的值不变）

`LDR R1, [R0, R2, LSL #02]`

;将R0+R2*4地址的数据读出，保存到R1中(R0、R2的值不变)

`LDR Rd, label`

;label为程序标号，label必须是当前指令-4~4KB范围内

3.3.3 存储器访问指令

- **LDRSB** R1, [R0, R3]

将R0+R3地址上的字节数据读到R1,高24位用符号位扩展。

- **LDRSH** R1, [R9]

将R9地址上的半字数据读出到R1, 高16位用符号位扩展。

- **LDRH** R6, [R2], #2

将R2地址上的半字数据读出到R6, 高16位用零扩展, 然后修改 $R2=R2+2$ 。

- **STRH** R1, [R0, #2]!

将R1的数据保存到R0+2地址中, 只存储低2字节数据, 并且修改 $R0=R0+2$ 。

3.3.3 存储器访问指令

示例：利用LDR实现多分支程序跳转

.....

MOV R2,R2,LSL #2

LDR PC,[PC,R2]

若此条指令为第一条指令，
则此时PC是第三条指令

NOP 占位

FUN_TAB	DCD	FUN_SUB0	0	=0*4
	DCD	FUN_SUB1	4	=1*4
	DCD	FUN_SUB2	8	=2*4
			12	=3*4
				

3.3.3 存储器访问指令

3.3.3.2 多寄存器的存取指令

LDM和STM指令可以实现在一组寄存器和一块连续的内存单元之间存/取数据。

LDM为加载多个寄存器；STM为存储多个寄存器。

这两条指令，允许传送16个寄存器R0---R15的任何子集或所有寄存器。

1、指令格式

LDM{cond}<模式> Rn{!}, <reglist>{^}

STM{cond} <模式> Rn{!}, <reglist>{^}

3.3.3 存储器访问指令

2、指令格式说明

(1) Rn: 表示基址寄存器，装有传送数据的初始地址，Rn不允许为R15（即PC）。

(2) Rn后缀“!”：表示最后的地址写回到Rn中。

(3) Reglist: 表示寄存器列表，其中包含一个或多个寄存器。当寄存器不连续时，中间使用“,” 隔开。

格式例子：{R1, R2, R6-R9}

列表寄存器和存储器地址的关系规则：编号低的寄存器对应于存储器中低地址单元，编号高的寄存器对应于存储器中高地址单元。

3.3.3 存储器访问指令

(4) 后缀 “^” 说明

- 寄存器列表不包含PC：使用后缀 “^” 进行数据传送时，加载 / 存储的是用户模式的寄存器，而不是当前模式的寄存器。
- 寄存器列表包含有PC：除了正常的多寄存器传送外，还要将SPSR拷贝到CPSR中。该用法可用于异常处理返回。
- 禁用情况：后缀 “^” 不允许在用户模式或系统模式下使用。因为它们没有SPSR

3.3.3 存储器访问指令

(5) 当Rn在寄存器列表中且使用后缀 “!”

对于STM指令，若Rn为寄存器列表中的最低序号的寄存器，则会将Rn的初值保存；

其它情况下Rn的编译无法通过。

(6) 地址字对齐

这些指令寻址是字对齐的，即忽略地址位[1:0]。

3.3.3 存储器访问指令

指令编码格式

31	28	27	26	25	24	23	22	21	20	19	16	15	0
cond		1	0	0	P	U	S	W	L	Rn	register list		

其中，registerlist为寄存器列表，b0位与R0对应，b15位与R15对应。P、U、W用于区别不同的地址模式。P表示前/后变址（before/after），U表示地址增/减(I/D)，W表示回写。S用于恢复CPSR和强制用户位。当PC寄存器包含在LDM指令的reglist中，且S为1时，则当前模式的SPSR将被复制到CPSR，成为一个原子的返回和恢复状态指令；若reglist不包含PC寄存器，且S为1时，则加载/存储的是用户模式的寄存器。L用于区别加载(L为1)或存储(L为0)。Rn为基址寄存器。

3.3.3 存储器访问指令

(7) 关于模式项

LDM/STM的主要用途是现场保护、数据复制和参数传送等。其模式有如下8种（前面4种用于数据块的传输（为存储操作），后面4种是堆栈操作）：

类型	每次基址寄存器的操作	传送起始地址	Rn序号的变化
IA	先传送数据，后基地址加4	(Rn)	增加
IB	先基地址加4，后传送数据	(Rn) +4	增加
DA	先传送数据，后基地址减4	(Rn)	减少
DB	先基地址减4，后传送数据	(Rn) -4	减少

3.3.3 存储器访问指令

类型	堆栈类型	弹出 (pop) 指令	压入 (push) 指令
FA	递增满堆栈	LDMFA	STMFA
FD	递减满堆栈	LDMFD	STMFD
EA	递增空堆栈	LDMEA	STMEA
ED	递减空堆栈	LDMED	STMED

3.3.3 存储器访问指令

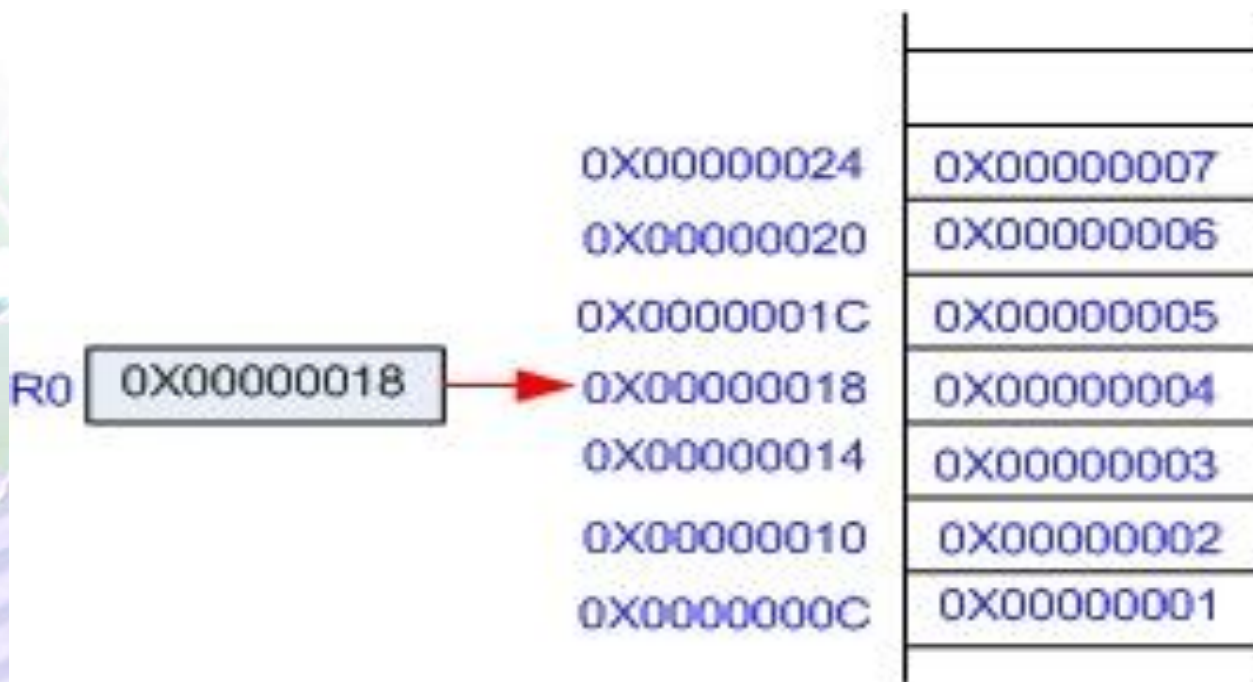
堆栈操作与批量传输的对应

寻址方式	说明	pop	=LDM	push	=STM
FA	递增满	LDMFA	LDMDA	STMFA	STMIB
FD	递减满	LDMFD	LDMI A	STMFD	STMDB
EA	递增空	LDMEA	LDMDB	STMEA	STMIA
ED	递减空	LDMED	LDMI B	STMED	STMDA

3.3.3 存储器访问指令

3、应用举例

例：数据在存储器中的存储情况如下图所示。现已知作为基地址寄存器 R0 的内容为 0X00000018，试分析如下指令执行后的结果。



3.3.3 存储器访问指令

(1) LDMIA R0! , {R1—R3}

解:

(R1) = 0X00000004

(R2) = 0X00000005

(R3) = 0X00000006

(R0) = 0X00000024



3.3.3 存储器访问指令

(2) LDMIB R0! , {R1—R3}

解:

(R1) = 0X00000005

(R2) = 0X00000006

(R3) = 0X00000007

(R0) = 0X00000024

R0 0X00000018

0X00000024

0X00000020

0X0000001C

0X00000018

0X00000014

0X00000010

0X0000000C

0X00000007

0X00000006

0X00000005

0X00000004

0X00000003

0X00000002

0X00000001

3.3.3 存储器访问指令

(3) LDMDA R0! , {R1—R3}

解:

(R3) = 0X00000004

(R2) = 0X00000003

(R1) = 0X00000002

(R0) = 0X0000000C



3.3.3 存储器访问指令

(4) LDMDB R0! , {R1—R3}

解:

(R3) = 0X00000003

(R2) = 0X00000002

(R1) = 0X00000001

(R0) = 0X0000000C

R0 0X00000018

0X00000024

0X00000020

0X0000001C

0X00000018

0X00000014

0X00000010

0X0000000C

0X00000007

0X00000006

0X00000005

0X00000004

0X00000003

0X00000002

0X00000001

3.3.3 存储器访问指令

STMFD SP!, {R0 - R7, LR}

；现场保存，将R0~R7、LR入栈，SP值更新。

LDMFD SP!, {R0 - R7, PC}^

；恢复现场，包括CPSR，异常处理返回，SP值更新。

3.3.3 存储器访问指令

3.3.3.3 单寄存器交换指令 (SWP)

SWP指令用于将一个存储单元(该单元地址放在寄存器Rn中)的内容读取到一个寄存器Rd中，同时将另一个寄存器Rm的内容写入到该存储单元中。

交换指令是一个原子操作，也就是说，在连续的总线操作中读/写一个存储单元，在操作期间阻止其它任何指令对该存储单元的读写。

3.3.3 存储器访问指令

1、指令格式

SWP{cond}{B} Rd, Rm, [Rn]

- B为可选后缀，若有B，则交换无符号字节，否则交换32位字；
- Rd为被加载的寄存器；
- Rm的数据用于存储到Rn所指的地址中，若Rm与Rd相同，则为寄存器与存储器内容进行交换；
- Rn为要进行数据交换的存储器地址，Rn不能与Rd和Rm相同。
- 功能：将一个内存单元[Rn]的内容读取到一个寄存器Rd中，同时将另一个寄存器Rm的内容写入到该内存单元中。

3.3.3 存储器访问指令

指令编码格式

31	28	27	26	25	24	23	22	21	20	19	16	15	12	11	4	3	0				
cond		0	0	0	1	0	B	0	0	Rn	Rd		0	0	0	0	1	0	0	1	Rm

其中，B用于区别无符号字节(B为1)或字(B为0)。Rm为源寄存器，Rd为目标寄存器，Rn为基址寄存器。

3.3.3 存储器访问指令

2、指令举例

SWP R1, R1, [R0]

；将R1的内容与R0指向的存储单元的内容进行交换。

SWPB R1, R2, [R0]

；将R0指向的存储单元的内容读取1字节数据到R1中(高24位清零)，并将R2的内容写入到该内存单元中(最低字节有效)

3.3.4 协处理器指令（了解）

ARM协处理器： ARM支持16个协处理器，用于各种协处理器操作，最常使用的协处理器是用于控制片上功能的系统协处理器，例如控制高速缓存和存储器的管理单元，浮点ARM协处理器等，还可以开发专用的协处理器。

ARM协处理器指令根据其用途主要分为以下三类：

- 协处理器数据操作指令。
- ARM寄存器与协处理器寄存器的数据传送指令。
- 协处理器寄存器和内存单元之间数据存/取指令。

3.3.4 协处理器指令（了解）

ARM协处理器指令包括以下5条：

- CDP 协处理器数操作指令 coprocessor;
- LDC 协处理器数据加载指令；
- STC 协处理器数据存储指令；
- MCR ARM处理器寄存器到协处理器寄存器的数据传送指令；
- MRC 协处理器寄存器到ARM处理器寄存器的数据传送指令。

3.3.4 协处理器指令（了解）

一、CDP——协处理器数据操作指令

ARM处理器通过CDP指令通知ARM协处理器执行特定的操作。协处理器数据操作完全是协处理器内部的操作，用于初始化ARM协处理器，完成协处理器寄存器的状态改变。

指令格式：CDP{<cond>} <CP#>, <Cop1>, CRd,
CRn, CRm {,<Cop2>}

- CP# 指令操作的协处理器名。标准名为pn, n为0~15。
- Cop1 协处理器的特定操作码。
- CRd 作为目标寄存器的协处理器寄存器。

3.3.4 协处理器指令（了解）

- CRn 存放第1个操作数的协处理器寄存器。
- CRm 存放第2个操作数的协处理器寄存器。
- Cop2 可选的协处理器特定操作码。

指令举例如下：

CDP p7, 0, c0, c2, c3, 0

；协处理器7执行操作码1为0和可选操作码2为0的操作。

CDP p6, 1, c3, c4, c5

；协处理器6执行；操作码为1的操作

3.3.4 协处理器指令（了解）

指令特点：

该操作由协处理器完成，即对命令参数的解释与协处理器有关，指令的使用取决于协处理器。

若协处理器不能成功地执行该操作，将产生未定义指令异常中断。

3.3.4 协处理器指令（了解）

二、LDC/STC——协处理器数据取/存指令

协处理器数据取/存指令从存储器读取数据装入协处理器寄存器，或将协处理器寄存器的数据存入存储器。

1、LDC——协处理器数据读取指令

LDC指令从某一连续的内存单元将数据读取到协处理器的寄存器中。进行协处理器的数据传送时，由协处理器来控制传送的字数。若协处理器不能成功地执行该操作，将产生未定义指令异常中断。

3.3.4 协处理器指令（了解）

指令格式如下：

LDC{cond}{L} <CP#>, CRd, <地址>

- L 可选后缀，指明是长整数传送。
- CP# 指令操作的协处理器名。标准名为pn，n为0~15。
- CRd 作为目标寄存的协处理器寄存器。
- <地址> 指定的内存地址。

指令举例如下：

LDC p5, c2, [R2, #4] ; 读取R2+4指向的内存单元的数据，传送到协处理器p5的c2寄存器中。

LDC p6, c2, [R1] ; 读取R1指向的内存单元的数据，传送到协处理器p6的c2寄存器中。

3.3.4 协处理器指令（了解）

2、STC——协处理器数据存入指令

将协处理器的寄存器数据存入到某一连续的内存单元中，由协处理器来控制写入的字数。

若协处理器不能成功地执行该操作，将产生未定义指令异常中断。

指令格式：STC{cond}{L} CP#, CRd, <地址>

其中格式说明同 LDC 指令。

指令举例如下：

STC p5, c1, [R0]

STC p5, c1, [R0, #0x04]

3.3.4 协处理器指令（了解）

三、ARM寄存器与协处理器寄存器的数据传送指令

1、MCR ——ARM寄存器到协处理器寄存器的数据传送指令

MCR指令将ARM处理器的寄存器中的数据传送到协处理器的寄存器中。

若协处理器不能成功地执行该操作，将产生未定义指令异常中断。

指令格式如下：

MCR{cond} CP# , Cop1, Rd, CRn, CRm{, <Cop2>}

其中格式说明同 CDP 指令。

3.3.4 协处理器指令（了解）

三、ARM寄存器与协处理器寄存器的数据传送指令

1、MCR ——ARM寄存器到协处理器寄存器的数据传送指令

Cop1：协处理器的特定操作码。对于CP15寄存器来说，
Cop1=0

Rd：源ARM寄存器，其值将被传送到协处理器寄存器中。

CRn：目标寄存器的协处理器寄存器，其编号是C~C15。

CRm：协处理器中附加的目标寄存器操作数寄存器。如果不需要设置附加信息，将CRm设置为c0，否则结果未知

Cop2：可选的协处理器特定操作码。（用来区分同一个编号的不同物理寄存器，当不需要提供附加信息时，指定为0）

3.3.4 协处理器指令（了解）

三、ARM寄存器与协处理器寄存器的数据传送指令

1、MCR ——ARM寄存器到协处理器寄存器的数据传送指令

例：

MCR p14, 0, Rn, c1, c0

；把ARM寄存器Rn中的值，写入到协处理器P14的C1寄存器中。

MCR p15,0,r3,c2,c0

；把ARM寄存器R3中的值，写入到协处理器P15的C2寄存器中。

3.3.4 协处理器指令（了解）

2、MRC——协处理器寄存器到ARM寄存器的数据传送指令

MRC指令将协处理器寄存器中的数据传送到ARM处理器的寄存器中。若协处理器不能成功地执行该操作，将产生未定义指令异常中断。

指令格式如下：

MRC{cond} CP# , Cop1, Rd, CRn, CRm{, <Cop2>}

其中格式说明同 MCR 指令。

指令举例如下：

MRC p5, 2, R2, c3, c2。

3.3.4 协处理器指令（了解）

2、MRC——协处理器寄存器到ARM寄存器的数据传送指令
例：

MRC p15, 0, <Rd>, c0, c0, 0

;返回协处理器C0寄存器中的32位的设备ID号到ARM Rd中。

MRC p15, 0, <Rd>, c0, c0, 1

;返回协处理器C0寄存器中的caches的信息到ARM Rd中。

MRC p15, 0, <Rd>, c2, c0, 0

;返回协处理器C2寄存器中的值到ARM Rd中。

P15 有2个C0寄存器：ID号寄存器 和 缓存类型寄存器

3.3.5 杂项指令

主要由两种类型指令组成，程序状态寄存器操作指令、中断操作指令，一共有5条指令。

➤ 状态寄存器操作指令：

- MRS：读程序状态寄存器指令
- MSR：写程序状态寄存器指令

➤ 异常中断操作指令：

- SWI：软件中断指令
- BKPT：断点指令（v5T体系）
- CLZ：前导0计数（v5T体系）

3.3.5 杂项指令

3.3.5.1 程序状态寄存器处理指令

ARM指令中有两条指令，用于在状态寄存器和通用寄存器之间传送数据。修改状态寄存器一般是通过“读取 - 修改 - 写回”三个步骤的操作来实现的。

1、MRS——读状态寄存器指令

指令格式：MRS{cond} Rd, psr ; $Rd \leftarrow psr$

把状态寄存器psr（CPSR或SPSR）的内容传送到目标寄存器中。

其中：Rd —— 目标寄存器。Rd不允许为R15。

psr —— CPSR或SPSR。

3.3.5 杂项指令

指令编码格式

[illegible]

其中，R用于区别CPSR(R为0)或SPSR(R为1)。

3.3.5 杂项指令

注意：在ARM处理器中，只有MRS指令可以将状态寄存器CPSR或SPSR读出到通用寄存器中。

指令举例如下：

MRS	R1, CPSR	; R1 <- CPSR
MRS	R2, SPSR	; R2 <- SPSR

3.3.5 杂项指令

2、MSR——写状态寄存器指令

在ARM处理器中，只有MSR指令可以直接设置状态寄存器CPSR或SPSR。

指令格式如下：

MSR{cond} psr_fields, #immed

MSR{cond} psr_fields, Rm

其中：

- psr: CPSR或SPSR。
- immed: 要传送到状态寄存器指定域的8位立即数。
- Rm: 要传送到状态寄存器指定域的数据的源寄存器。

3.3.5 杂项指令

- fields 指定传送的**区域**。fields可以是以下的一种或多种：
 - c 控制域 (psr[7...0]);
 - x 扩展域(psr[15...8]); (暂未用)
 - s 状态域 (psr[23...16]); (暂未用)
 - f 标志位域 (psr[31...24])。

程序状态寄存器分域图。



3.3.5 杂项指令

指令举例如下：

MSR CPSR_cxsf, R0 ; 传送R0的内容到CPSR

MSR SPSR_cxsf, R0 ; 传送R0的内容到SPSR

MSR CPSR_c, R0

; 传送R0的内容到CPSR, 但仅仅修改CPSR中的控制位域

MSR CPSR_cfxs, R0

; 传送R0的内容到CPSR, 修改所有域

CPSR的读—修改—写操作举例如下：

例1：设置进位位C

MRS R0, CPSR

ORR R0, R0, #0x20000000

MSR CPSR_f, R0

;R0<- CPSR

;置1进位位C

;CPSR_f<-R0[31:24]

3.3.5 杂项指令

注 意：

- 控制域的修改问题：只有在特权模式下才能修改状态寄存器的控制域[7：0]，以实现处理器模式转换，或设置开/关异常中断。
- T控制位的修改问题：程序中不能通过MSR指令，直接修改CPSR中的T控制位来实现ARM状态 / Thumb状态的切换，必须使用BX指令完成处理器状态的切换。
- 用户模式下能够修改的位：在用户模式只能修改“标志位域”，不能对CPSR[23：0]做修改。
- S后缀的使用问题：在MRS/MSR指令中不可以使用S后缀。

3.3.5 杂项指令

例：堆栈初始化

INITSTACK		；堆栈初始化子程序
MOV R0, LR		；保存返回地址
MRS R1, CPSR		；保存CPSR原值
MSR CPSR_c, #0xD7		；切换到中止模式
.....		；设置堆栈大小等
LDR SP, StackSvc		；设置堆栈指针
MSR CPSR_c, #0xD2		；切换到中断模式
.....		；设置堆栈大小等
LDR SP, StackIrq		；设置堆栈指针
.....		；对其它模式设置
MSR CPSR_c, R1		；恢复CPU原模式
MOV PC, R0		；子程序返回

3.3.5 杂项指令

3.3.5.2 异常中断产生指令

异常中断指令可以分为以下几种：

- SWI： 软件中断指令
- BKPT： 断点指令（v5T及以上体系）
- CLZ： 前导0计数（v5T及以上体系）

3.3.5 杂项指令

1、SWI——软件中断指令

软件中断指令SWI产生软件异常中断，用来实现用户模式到特权模式的切换。

- 用于在用户模式下对操作系统中特权模式的程序的调用；
- 它将处理器置于管理（svc）模式，中断矢量地址为0x08。

指令格式如下：SWI {<cond>} <24位立即数>

24位立即数，指定用户程序调用系统例程的类型，相关参数通过寄存器传递，当指令中24位立即数被忽略时(立即数为0)，用户程序调用系统例程的类型由通用寄存器R0决定，同时参数通过其它寄存器传递

3.3.5 杂项指令

指令编码格式

31	28	27	26	25	24	23	0
cond		1	1	1	1	immed_24	

指令编码格式

3.3.5 杂项指令

说明:

- 主要用于用户程序调用操作系统的API。
- 参数传递通常有两种方法:
 - 指令中的24bit立即数指定API号, 其它参数通过寄存器传递。
 - 忽略指令中的24bit立即数, r0指定API号, 其它参数通过其它寄存器传递。

3.3.5 杂项指令

SWI指令举例：

软中断号在指令中，**不传递**其它参数：

SWI 10 ; 中断类型为10 (**注：没有#号**)

SWI 0x123456 ; 中断类型为0x123456

软中断号在指令中，**其它参数**在寄存器中传递：

MOV R0, #34 ; 准备参数

SWI 12 ; 调用12号软中断

不用指令中的立即数，软中断类型号和其它参数都在寄存器中传递：

MOV R0, #12 ; 准备中断类型号

MOV R1, #34 ; 准备参数

SWI 0 ; 进入软中断。

3.3.5 杂项指令

SWI程序举例：进入SWI中断程序后，如何提取中断类型码。

T_bit EQU 0x20 ; 用于测试Thumb标志位(bit5)

SWI_handler

STMFD SP! , {R0-R3, R12, LR}

MRS R0, SPSR ; 中断前的CPSR值到R0

TST R0, #T_bit ; 测试T位标志

LDRNEH R0, [LR, #-2] ; 读取16位的SWI指令码

BICNE R0, R0, #0xFF00 ; 获取SWI中的中断号

LDREQ R0, [LR, #-4] ; 读取32的SWI指令码

BICEQ R0, R0, #0xFF000000 ; 获取SWI中的中断号

..... ; 转去处理相应的软中断

LDMFD SP! , {R0-R3, R12, PC}

; 中断返回，包括恢复原CPSR值

2.9 ARM的中断（异常）

回顾

ARM处理器进入异常时R14所保存的值及退出时推荐指令：

异常类型	返回指令	之前的状态		备注
		ARM R14_x	Thumb R14_x	
BL	MOV PC,R14	$PC_{\text{寄}} = \text{Adr}_{\text{执}} + 4$	PC+2	此处PC为BL，SWI，未定义的指令取指或预取指中止指令的地址
软件中断 SWI	MOVS PC,R14_svc	$PC_{\text{寄}} = \text{Adr}_{\text{执}} + 4$	PC+2	
未定义的指令	MOVS PC,R14_und	$PC_{\text{寄}} = \text{Adr}_{\text{执}} + 4$	PC+2	
预取指中止	SUBS PC,R14_abt,#4	$PC_{\text{寄}} = \text{Adr}_{\text{执}} + 4$	PC+4	
快中断	SUBS PC,R14_fiq,#4	$PC_{\text{寄}} = \text{Adr}_{\text{执}} + 8$	PC+4	此处PC为由于FIQ或IRQ占先而没有被执行的指令的地址
中断	SUBS PC,R14_irq,#4	$PC_{\text{寄}} = \text{Adr}_{\text{执}} + 8$	PC+4	
数据中止	SUBS PC,R14_abt,#8	$PC_{\text{寄}} = \text{Adr}_{\text{执}} + 8$	PC+8	此处PC为产生数据中止的装载或保存指令的地址。
复位 184	无	—	—	复位时保存在R14_svc中的值不可预知。

3.3.5 杂项指令

2、BKPT——断点指令（了解）

断点中断指令BKPT用于产生软件断点，供调试程序用。
v5T及以上体系使用。

指令格式如下： BKPT {immed_16}

immed_16：16位的立即数。该立即数被调试软件用来保存额外的断点信息。

断点指令用于软件调试；它使处理器停止执行正常指令而进入相应的调试程序。

3.3.5 杂项指令

3、CLZ——前导0计数指令（了解）

前导0计数指令CLZ 对Rm中的前导0的个数进行计数，结果放到Rd中。 v5T及以上体系使用。

指令格式：CLZ{<cond>} Rd, Rm

举例如下：

MOV R2, #0X17C00

; R2=0b0000 0000 0000 0001 0111 1100 0000 0000

CLZ R3, R2 ; R3=15

3.3.6 例题

一、算术逻辑运算指令的应用

例1：实现乘法的指令段

MOV R0,R0,LSL #n ;R0=R0<<n; R0= R0*2ⁿ

ADD R0,R0,R0,LSL #n ;R0=R0+R0*2ⁿ= R0*(2ⁿ+1)

RSB R0,R0,R0,LSL #n ;R0=R0*2ⁿ-R0= R0*(2ⁿ-1)

3.3.6 例题

例2：64位数据运算

假设R0和R1存放一个64位数据，R0中存放数据的低32位；R2和R3中存放另一个64位数据，R2中存放数据的低32位。对这两个64位数进行加、减和比较运算。（R1, R0）、（R3, R2）

①两个64位数据的加法运算，结果保存到R0和R1中。

ADDS R0,R0,R2 ;低32位相加，设置CPSR的C标志位。

ADC R1,R1,R3 ;高32位的带位相加

3.3.6 例题

②两个64位数据的减法运算，结果保存到R0和R1中。

SUBS R0,R0,R2 ;低32位相减，设置CPSR的C标志位。

SBC R1,R1,R3 ;高32位的带位相减

③两个64位数据的比较操作，并设置CPSR中的条件标志位。

CMP R1,R3 ;比较高32位

CMPEQ R0,R2 ;如果高32位相等，比较低32位

3.3.6 例题

例3：转换内存中数据存储方式

将寄存器R0中的数据存储方式转换成另一种存储方式。

指令执行前R0中数据存储方式为：R0=A,B,C,D；指令执行后R0中数据存储方式为：R0=D,C,B,A。

EOR R1,R0,R0, ROR #16 ;R1=A^C,B^D,C^A,D^B

BIC R1,R1,#0xFF0000 ;R1=A^C,0,C^A,D^B

MOV R0,R0,ROR #8 ;R0=D,A,B,C

EOR R0,R0,R1,LSR #8 ;R0=D,C,B,A

;R0=D,A,B,C异或R1=0,A^C,0,C^A

3.3.6 例题

二、跳转指令的应用

例1：子程序的调用

BL指令在执行跳转操作的同时保存下一条指令的地址，用于从被调用的子程序中返回。

```
.....  
BL  function ;调用子程序function  
                        ;子程序结束后，程序将返回到这里执行  
.....  
.....  
Function ;子程序的程序体  
.....  
MOV PC,LR ;子程序中的返回语句
```

3.3.6 例题

例2：条件执行

实现类似于C语言中的if-else功能的代码段。下例的功能为求最大公约数。

C语言代码为：

```
int gcd (int a, int b)
{
    while (a!=b)
    {   if (a>b)  a=a-b;
        else    b=b-a;
    }
    return a;
}
```

3.3.6 例题

对应的ARM汇编代码段。（代码执行前R0中存放a，R1中存放b；代码执行后R0中存放最大公约数。

gcd

```
CMP    R0,R1    ;比较a和b的大小
SUBGT  R0,R0,R1 ;if(a>b) a=a-b
SUBLT  R1,R1,R0 ;if(b>a) b=b-a
BNE    gcd      ;if(a!=b)跳转到gcd继续执行
MOV    PC,LR    ;子程序结束，返回
```


3.3.6 例题

循环结构

```
MOV R1,#10
```

```
LOOP ..... ;循环体
```

```
SUBS R1,R1,#1
```

```
BNE LOOP
```

```
.....
```

例3：循环语句

将内存中从0x400800开始的100个字数据相加，其结果存于R3、R2中（R3中为高32位）。

3.3.6 例题

LDR R0, =0x400800

MOV R1, #100 ;初始化循环次数

MOV R2, #0

MOV R3, #0

loop ;循环体

LDR R4, [R0], #4

ADDS R2, R2, R4

ADC R3, R3, #0 ; 或者ADDCS R3, R3, #1

SUBS R1, R1, #1; 循环计数器减1, 设置条件标志

BNE loop ;循环计数器不为0,跳到loop继续执行

..... ;循环计数器为0, 程序继续执行

3.3.6 例题

例4：链表操作中的条件码应用

在链表中搜索与某一数据相等的元素。

链表的每个元素包括两个字，第1个字中包含一个字节数据；第2个字中包含指向下一个链表元素的指针，当这个指针为0时表示链表结束。

代码执行前R0指向链表的头元素，R1中存放将要搜索的数据；代码执行后R0指向第1个匹配的元素，或者当没有匹配元素时，R0为0。

3.3.6 例题

SEARCH

CMP	R0,#0	;R0指针是否为空
LDRNEB	R2,[R0]	;读取当前元素中的字节数据
CMPNE	R1,R2	;判断数据是否为搜索的数据
LDRNE	R0,[R0,#4]	;如果不是,指针R0指向下一个元素
BNE	SEARCH	;跳转到search执行
MOV	PC,LR	;搜索完成, 程序返回

作业

作业见群文件《平时作业要求》中第3章的内容。